

Gym Trainer Model



Project ID: Fall-2022-2026

Session: BSCS Fall 2022 to 2026

Project Advisor: Sir Salman Ali

Submitted By

Muhammad Hassan	70138485
Muhammad Adil Nadeem	70139389
Muneeb Waseem	70139133

Department of Computer Science & IT
The University of Lahore
Lahore, Pakistan

Declaration

We have read the project guidelines and we understand the meaning of academic dishonesty, in particular plagiarism and collusion. We hereby declare that the work we submitted for our final year project, entitled **Gym Trainer Model** is original work and has not been printed, published or submitted before as final year project, research work, publication or any other documentation.

Group Member 1 Name: Muhammad Hassan

SAP No: 70138485

Signature:

Group Member 2 Name: Muhammad Adil Nadeem

SAP No: 70139389

Signature:

Group Member 3 Name: Muhammad Muneeb Waseem

SAP No: 70139133

Signature:

Statement of Submission

This is to certify that **Muhammad Hassan** Roll No. **70138485**, **Muhammad Adil Nadeem** Roll No. **70139389** and **Muneeb Waseem** Roll No. **70139133** have successfully submitted the final project named as: **Gym Trainer Model**, at Computer Science & IT Department, The University of Lahore, Lahore Pakistan, to fulfill the partial requirement of the degree of **BS in Computer Science**.

Supervisor Name:

Signature:

Date:

Dedication

I dedicate this project to Almighty Allah, whose countless blessings, guidance, and strength made the completion of this work possible. I also dedicate this project to my parents, whose unconditional love, prayers, sacrifices, and continuous encouragement have been the foundation of my academic journey. Special thanks to my respected teachers and supervisor for their valuable guidance, support, and motivation throughout this project. Lastly, I dedicate this work to all those who inspired me, supported me morally, and believed in my abilities during the completion of this final year project.

Acknowledgement

We truly acknowledge the cooperation and help make by **Sir Salman Ali, Lecturer at University of Lahore**. She has been a constant source of guidance throughout the course of this project. We would also like to thank **Sir Salman Ali, Lecturer at University of Lahore** for her help and guidance throughout this project. We are also thankful to our friends and families whose silent support led us to complete our project.

Date:

Dec 24,2025

Abstract

Physical exercises are a major part of a healthy life, but exercising with an improper posture and methods increases the risk of severe injuries and undesired outcomes of exercising. The aim of this project is to design an AI-based system called a Gym Trainer that will aid the users by checking their performance during the exercising process and giving an instant reaction without any human assistance. The challenge in the project is the lack of access to affordable real-time instruction for people conducting gym exercises, particularly in the context of home workouts. Many people are not aware that their body positioning, angles, and repetitions could potentially lead to muscle strain and health problems. The proposed system applies machine learning and computer vision models to detect video feed from a webcam. The poses of humans are identified by using a landmark-based model with calculation of angle positions for tracking exercises. The system also calculates repetitions in exercises, checks correctness, and indicates wrong repetitions. The system also indicates wrong repetitions in exercises with real-time feedback using voice commands or buzzer signals. The proposed system has to be implemented on a web platform or application. The intended outcomes include accurate repetition counting, accurate posture analysis, and accurate and timely feedback to enable users to correct exercise posture. The system should be capable of improving workout efficiency and preventing possible exercise injuries.

Area of the Project

Machine Learning, Artificial Intelligence, Computer Vision, Image Processing, Human Pose Estimation, Web Application Development

Technologies used

Programming Languages:

- Python (for model development and pose analysis)
- JavaScript (for web-based application integration)

Libraries and Frameworks:

- MediaPipe (for real-time human pose detection and landmark extraction)
- OpenCV (for video processing and angle calculation)
- Numpy

Compilers:

- Jupyter Notebook/Google Collab
- Visual Studio

Visual Paradigm:

- For creating UML diagrams (Use Case, DFD, Sequence, Deployment, Component)

GitHub:

- For version control and project backup.

List of Figures

Figure 11.	Architecture Diagram	43
Figure 12.	ERD with data dictionary	44
Figure 13.	Data Flow diagram	45
Figure 14.	Class Diagram	46
Figure 15.	Activity Diagram	47
Figure 16.	Sequence Diagram	48
Figure 17.	Collaboration Diagram	49
Figure 18.	State Transition Diagram	50
Figure 19.	Component Diagram	51
Figure 20.	Deployment Diagram	51

List of Tables

Functional Requirements Tables

Table 2.1 User Account & Authentication.....	11
Table 2.2 Body type & Fitness Profile.....	13
Table 2.3 Diet Plan Generation.....	14
Table 2.4 Exercise Selection & Guidance.....	15
Table 2.5 Live Exercise Monitoring(AI Module).....	16
Table 2.6 Progress Tracking	17
Table 2.7 Notifications & Reminders.....	18
Table 2.8 Admin Functionalities.....	19
Table 2.9 Security & Data Handling	20
Table 2.10 Exercise Difficulty Levels.....	21
Table 1 Functional Requirements of the AI-Based Gym Trainer System	35
Table 2 Non-Functional Requirements of the System	36
Table 3 Use Case Description – Exercise Monitoring and Feedback	37
Table 4 Input and Output Specifications	38
Table 5 Tools, Libraries, and Technologies Used	39
Table 6 Hardware and Software Requirements	40
Table 7 Supported Exercises and Joint Angle Thresholds	40
Table 8 Dataset Description and Pose Features	41
Table 9 Model Training and Performance Evaluation Metrics	42

Table of Content

Declaration	i,ii
Statement of Submission.....	iii
Dedication	iv
Acknowledgement	v
Abstract	vi
Area of the Project	vii
List of Figures	viii
List of Tables.....	ix
Chapter 1: Introduction to Problem	1
1.1 Introduction.....	1
1.2 Purpose	1
1.3 Objective	1
1.4 Existing Solution	2
1.5 Proposed Solution	2
Chapter 2: Software Requirement Specification	3
2.1 Introduction.....	3
2.1.1 Purpose	3
2.1.2 Scope.....	3
2.1.3 Definitions, Acronyms, and Abbreviations.....	4
2.2 Overall Description	5
2.2.1 Product Perspective	5
2.2.2 Product Functions	8
2.2.3 User Characteristics	22
2.2.4 Constraints	23
2.2.5 Assumptions and Dependencies	26

2.2.6 Apportioning of requirements	29
2.3 Specific requirements.....	30
2.3.1 Functional requirements.....	31
2.3.2 Non -functional Requirements	32
Chapter 3: Use Case Analysis	25
Chapter 4: Design	43
4.1 Architecture Diagram	43
4.2 ERD with data dictionary	44
4.3 Data Flow Diagram	45
4.3.1 The level 0	45
4.3.2 The level 1	45
4.4 Class Diagram	46
4.5 Activity Diagram	47
4.6 Sequence Diagram	48
4.7 Collaboration Diagram	49
4.8 State Transition Diagram.....	50
4.9 Component Diagram	51
4.10 Deployment Diagram	52
Chapter 5: User Manual	53

Chapter 1: Introduction to the Problem:

1.1 Introduction:

With the rising awareness regarding health, fitness, and exercise, many people are performing gym exercise either in fitness centers or in the comfort of their homes. However, one of the primary issues encountered by most users is carrying out the exercise incorrectly. Lack of proper angles in joints, extraordinary movement, and unrestricted repetition can lead to physical strain, injuries, and inefficiency in performing the exercise. With the trainer either not available or expensive, it is a common scenario to find users performing exercise on their own without any immediate guidance. The proposed project will resolve this problem by presenting an intelligent system capable of monitoring exercise performance.

1.2 Purpose:

The aim of developing this project is to design an affordable and accessible AI Gym Trainer that helps users properly do physical exercises. This project can prove to be beneficial for society because everyone will be able to work out properly and stay fit without having to spend extra money on trainers and gyms. Additionally, this project will especially help those individuals who work out at home and cannot afford professional trainers.

1.3 Objectives:

The primary aims for this project are:

- For recognizing and determining human body posture using a webcam in real time
- Calculation of joint angles involved in exercise movements
- The accurate calculation of exercise repetitions
- To recognize improper exercise posture and movements

- For real-time feedback through voice commands or buzzer alerts
- The task is to design a web-based or app-based interface for the users to engage with

1.4 Existing Solution:

The existing solution available in the marketplace includes a mobile fitness app, a video tutorial workout guide, and a smart wearable device. The majority of the fitness apps merely include pre-recorded videos and do not include real-time posture analysis. Wearable devices capable of tracking the repetition of the workout lack accurate posture analysis and are costlier. Neither of the mentioned solutions provides real-time corrections in the workout technique pertaining to the visual movements of the body.

1.5 Proposed Solution

The proposed solution is an AI-powered Smart Gym Trainer that tracks consumers using a webcam. By leveraging Python libraries like MediaPipe and OpenCV, the Smart Gym Trainer is able to identify body points, analyze joint positions, and track movements to identify the correctness of exercising. In case of incorrect posture, consumers are given immediate feedback in the form of voice notifications or buzzer sounds. In comparison to other solutions, this solution is inexpensive, offers immediate graphical feedback, and enables consumers to make necessary corrections instantly, thus being a better solution due to being a lot easier to use.

Chapter 2: Software Requirement Specification (SRS)

2.1 Introduction

2.1.1 Purpose

The purpose of this Software Requirement Specification (SRS) is to provide a comprehensive and detailed description of the AI-Based Gym Trainer system, covering its functionalities, constraints, expected behavior, and implementation guidelines. This document ensures that all stakeholders have a clear understanding of the project requirements and serves as a blueprint for development, testing, deployment, and maintenance.

Intended Audience:

- Software Developers and Programmers: For accurate implementation of system functionalities.
- Project Supervisors and Faculty Members: For reviewing, evaluation, and approval.
- End-Users: To understand system features, limitations, and usage.
- Future Maintenance Teams: For software updates and enhancements.

2.1.2 Scope:

The AI-Based Gym Trainer is a web-based application designed to monitor users performing gym exercises through a webcam and provide real-time feedback. The scope includes:

1. Software Product: AI-Based Gym Trainer Web Application.
2. Functional Capabilities:
 - Real-time human pose detection using MediaPipe and OpenCV.
 - Joint angle calculation to analyze exercise correctness.
 - Counting exercise repetitions.
 - Detecting incorrect posture and improper movements.
 - Providing corrective feedback via voice commands or buzzer alerts.
3. Non-Functional Capabilities:

- User-friendly web interface with interactive dashboards.
- Real-time performance with minimal processing delay.
- Secure storage of user profiles and exercise history.

4. Benefits:

- Promotes safe exercise habits and reduces the risk of injuries.
- Improves exercise efficiency and repetition accuracy.
- Makes professional-level gym guidance accessible at home.

5. Limitations:

- The system does not provide medical advice.
- Only supports predefined exercises included in the training dataset.

2.1.3 Definitions, Acronyms, and Abbreviations

2.1.3.1 Definitions:

- Exercise Repetition: One complete cycle of a particular exercise.
- Posture Analysis: Evaluation of body alignment and joint angles.
- Landmark: A key body point used for pose estimation (e.g., elbow, knee, shoulder).
- Repetition Counting: Process of tracking the number of correct exercise executions.
- Real-Time Feedback: Immediate guidance provided to the user during exercises.
- Pose Estimation: Machine learning technique to detect human body posture from video frames.

2.1.3.2 Acronyms:

- **AI:** Artificial Intelligence
- **ML:** Machine Learning
- **SRS:** Software Requirement Specification

- **UI:** User Interface
- **API:** Application Programming Interface
- **CV:** Computer Vision
- **DB:** Database
- **WebApp:** Web Application

2.1.3.3 Abbreviations:

- **UOL:** University of Lahore
- **FR:** Functional Requirement
- **ID:** Identifier (used for functional requirement IDs)
- **URL:** Uniform Resource Locator
- **GUI:** Graphical User Interface

These definitions, acronyms, and abbreviations ensure clarity and consistency throughout the SRS document, providing proper interpretation for all stakeholders.

2.2 Overall Description

2.2.1 Product Perspective:

The AI-Based Gym Trainer system is a **standalone and self-contained software application** that utilizes machine learning and computer vision techniques to analyze human exercise movements. Unlike traditional fitness applications that rely on pre-recorded videos or wearable sensors, this system performs **real-time visual analysis** using a standard webcam. The product can operate independently without requiring integration with external fitness systems, although optional extensions such as cloud storage or wearable integration may be added in future versions.

From a broader perspective, the system acts as a **virtual personal trainer**, offering features similar to professional gym guidance while remaining cost-effective and easily accessible. The major components of the system include webcam input, pose detection module, angle calculation module,

repetition counting logic, feedback mechanism, and web-based user interface. These components interact with each other to ensure smooth real-time operation.

1. System Interfaces:

The system interface defines how the software interacts with system-level components. The AI-Based Gym Trainer interfaces with:

- Webcam hardware to capture real-time video streams.
- Local or cloud-based storage to store user profiles and exercise data.
- Audio output devices for voice alerts and buzzer notifications.

The system processes video input frames, analyzes them using pose estimation models, and generates appropriate feedback based on predefined rules and thresholds.

2. User Interfaces:

The user interface is web-based and designed to be simple, intuitive, and user-friendly. It includes:

- A **home screen** for application access.
- An **exercise monitoring screen** displaying live webcam feed.
- Visual indicators for posture correctness and repetition count.
- Notifications and alerts for incorrect exercise performance.

The interface ensures minimal user interaction during workouts, allowing users to focus entirely on their exercises. Screenshots of the interface will be included in later chapters.

3. Hardware Interfaces:

The system requires minimal hardware components, including:

- A desktop or laptop computer.
- A built-in or external webcam for video capture.
- Audio output devices such as speakers or headphones.

No specialized hardware or wearable devices are required, making the system accessible and affordable.

4. Software Interfaces:

The AI-Based Gym Trainer depends on several software components:

- **Python** for core machine learning and computer vision processing.
- **MediaPipe** for real-time human pose detection.
- **OpenCV** for video frame processing and angle calculations.
- **JavaScript** for web application development and user interaction.
- A web browser for running the application interface.

5. Communications Interfaces

The system uses standard communication protocols, including:

- HTTP/HTTPS for web-based communication.
- Local system communication between frontend and backend modules.
- Internet connectivity for optional cloud services and updates.

6. Memory

The system requires sufficient primary memory (RAM) to process video frames and execute machine learning models in real time. Secondary storage is required for:

- Storing application files.
- Maintaining user profiles and exercise records.

Memory usage is optimized to ensure smooth performance without excessive resource consumption.

7. Operations

➤ **Normal Operations:**

- Launching the web application.
- Capturing live video feed.
- Real-time posture analysis and repetition counting.
- Generating feedback alerts.

➤ **Special Operations:**

- System startup and shutdown.
- Data backup and recovery.
- Model updates and configuration changes.

8. Site Adaptation Requirements

The system may require minor site-specific configurations, including:

- Webcam calibration and positioning.
- Environment lighting adjustments.
- Initialization of exercise thresholds based on user setup.

These adaptations ensure optimal performance across different environments and usage conditions.

2.2.2 Product Functions

The AI-Based Gym Trainer system provides a set of core product functions that collectively enable real-time exercise monitoring, posture analysis, and user feedback. These product functions represent the **functional requirements** of the system and define what the software is expected to perform. The major product functions are described below:

1. User Account Management

- Allows users to create and manage their accounts.
- Enables secure login and logout functionality.

- Stores basic user profile information for personalized exercise monitoring.

2. Webcam-Based Video Capture

- Captures real-time video stream from the user's webcam.
- Continuously processes video frames for pose detection.

3. Human Pose Detection

- Detects human body landmarks using machine learning models.
- Identifies key joints such as shoulders, elbows, hips, knees, and ankles.

4. Joint Angle Calculation

- Calculates angles between detected body joints.
- Uses angle thresholds to evaluate exercise correctness.

5. Exercise Recognition

- Identifies predefined exercises such as squats, push-ups, and bicep curls.
- Tracks movement patterns over time.

6. Repetition Counting

- Counts the number of correctly performed exercise repetitions.
- Resets and updates repetition count dynamically during workouts.

7. Incorrect Posture Detection

- Detects improper body posture or incorrect movement patterns.
- Compares real-time angles with predefined correct ranges.

8. Real-Time Feedback System

- Provides instant feedback when incorrect posture is detected.
- Generates voice alerts or buzzer notifications to guide the user.

9. Exercise Session Monitoring

- Displays live exercise progress, repetition count, and posture status.
- Allows users to start and stop exercise sessions.

10. Data Storage and History Management

- Stores exercise performance data for future reference.
- Maintains session history for user progress tracking.

These product functions ensure that the system operates as an intelligent virtual gym trainer, providing accurate analysis, timely feedback, and an improved workout experience. Each function will be further elaborated in the functional requirements section using detailed tables and workflows.

Functional Requirement Tables:

Table 2.1: User Account & Authentication

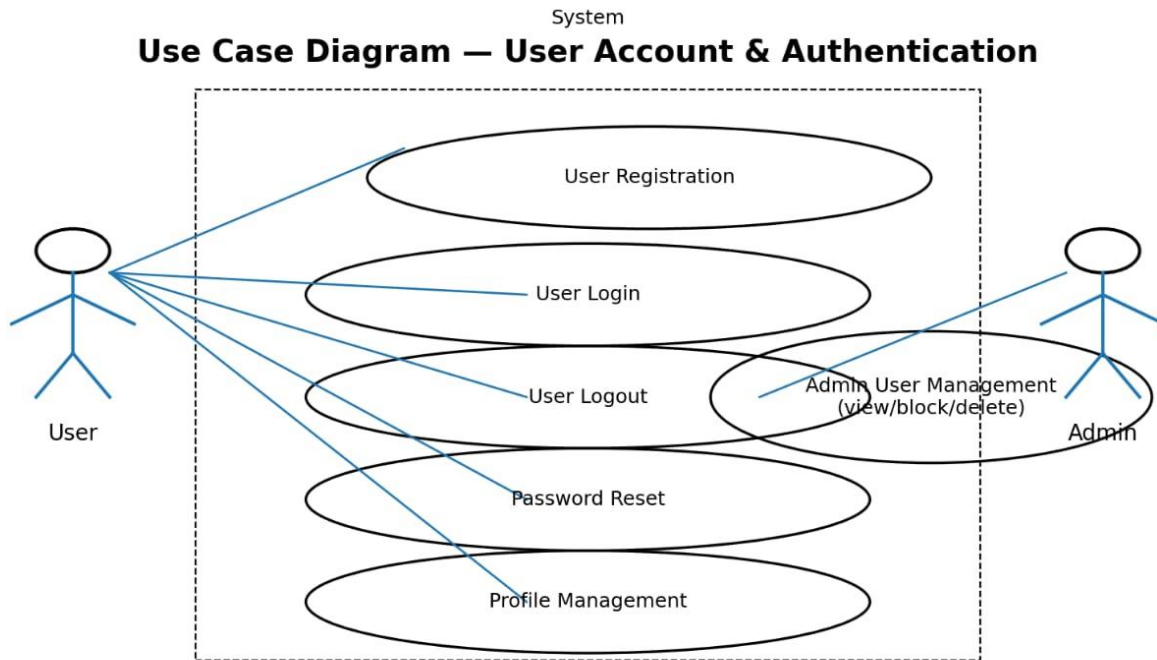


Figure 1: User Account & Authentication

Req ID	Functional Requirement	Description
FR-01	User Registration	The system shall allow users to register an account using email, username, and password.
FR-02	User Login	The system shall allow registered users to log in securely using valid credentials.
FR-03	User Logout	The system shall allow users to log out of the system securely.

FR-04	Password Reset	The system shall allow users to reset their password through email verification.
FR-05	Profile Management	The system shall allow users to update personal information such as age, height, weight, and fitness goals.
FR-06	Admin User Management	The system shall allow admin users to view, block, and delete user accounts.

Table 2.2: Body Type & Fitness Profile

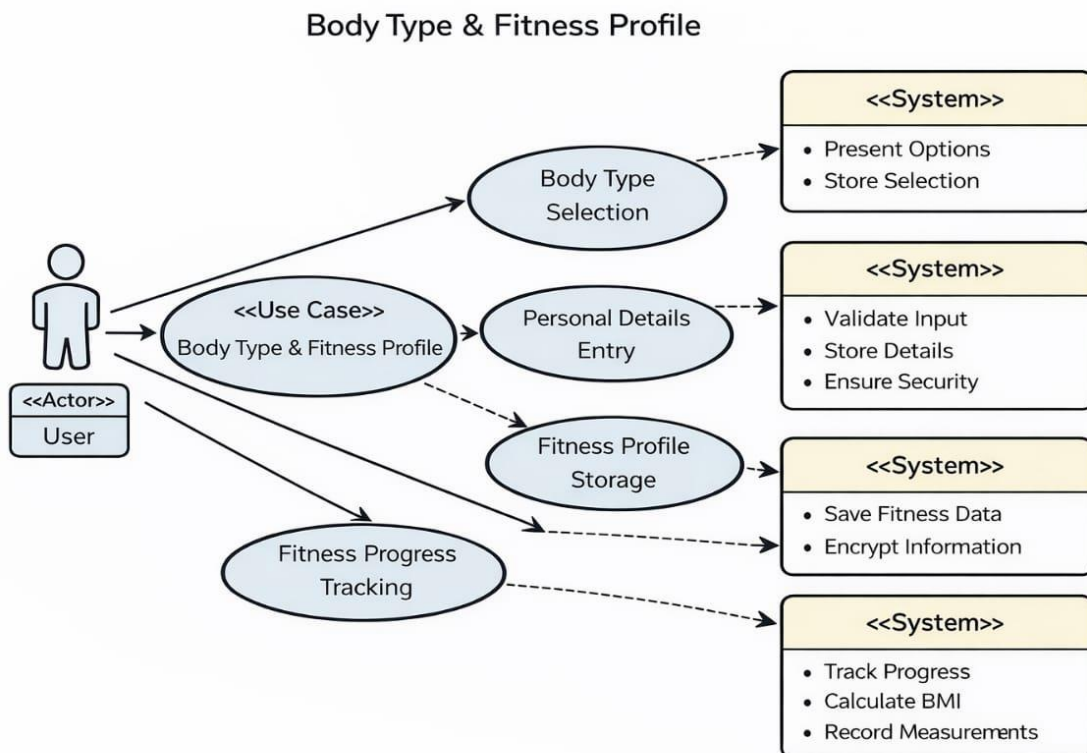


Figure 2: Body type & Fitness Profile

Req ID	Functional Requirement	Description
FR-07	Body Type Selection	The system shall allow users to select their body type, including ectomorph, mesomorph, endomorph, or a custom type.
FR-08	Personal Details Entry	The system shall allow users to enter personal details such as age, weight, height, and medical conditions.
FR-09	Fitness Profile Storage	The system shall store user fitness data securely for future reference.
FR-10	Fitness Progress Tracking	The system shall track and display fitness progress over time, including weight, BMI, and body measurements.

Table 2.3: Diet Plan Generation

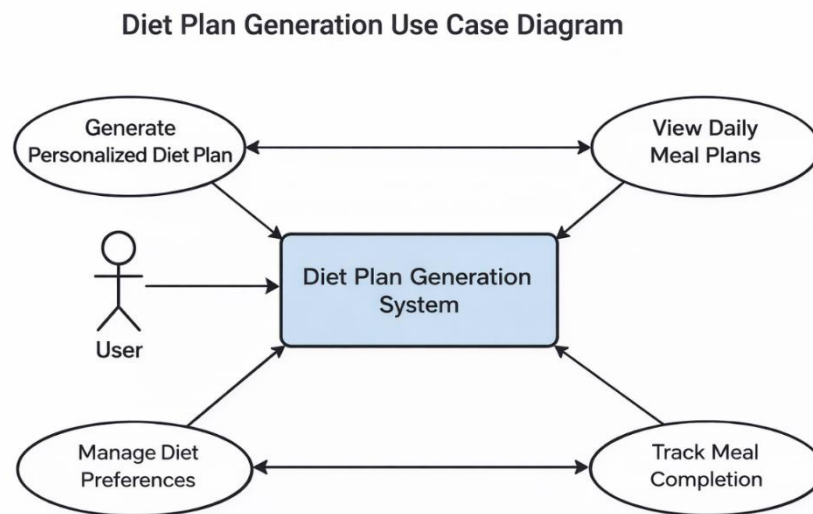


Figure 3: Diet Plan Generation

Req ID	Functional Requirement	Description
FR-11	Personalized Diet Plan	The system shall generate a personalized diet plan using AI based on the user's body type, fitness goals, and daily calorie requirements.
FR-12	Daily Meal Plan View	The system shall allow users to view structured daily meal plans including breakfast, lunch, dinner, and snacks.
FR-13	Diet Preference Management	The system shall allow users to update diet preferences such as vegetarian, vegan, non-vegetarian, and food allergies.
FR-14	Meal Completion Tracking	The system shall allow users to mark meals as completed to track diet adherence over time.

Table 2.4: Exercise Selection & Guidance

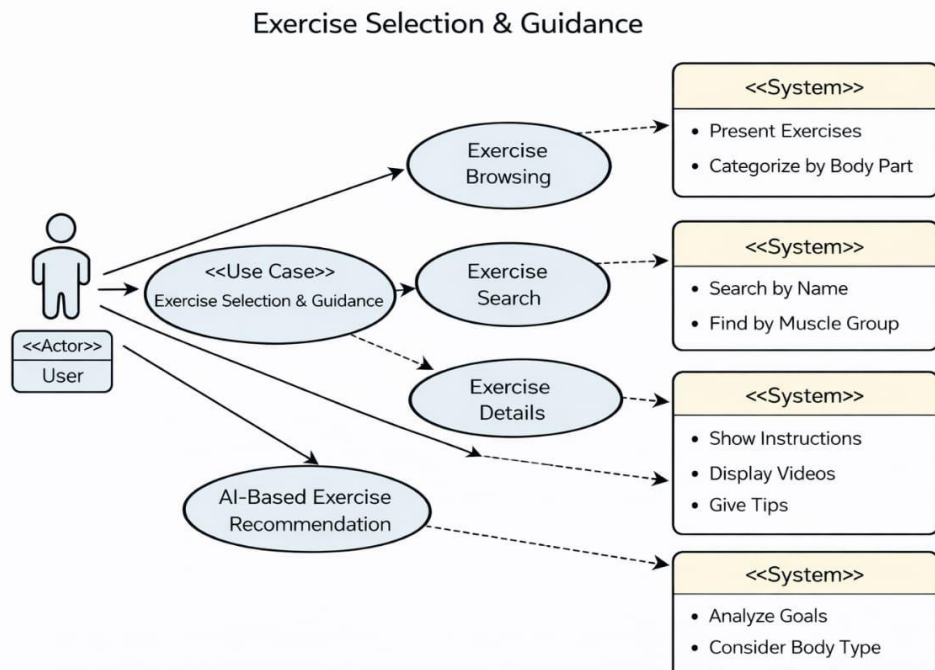


Figure 4: Exercise Selection & Guidance

Req ID	Functional Requirement	Description
FR-15	Exercise Browsing	The system shall allow users to browse a list of exercises categorized by body parts such as arms, legs, abs, and back.
FR-16	Exercise Search	The system shall allow users to search exercises by name or target muscle group.
FR-17	Exercise Details	The system shall display exercise details including instructions, demonstration videos/images, and performance tips.
FR-18	AI-Based Exercise Recommendation	The system shall recommend suitable exercises using AI based on the user's fitness goals and body type.

Table 2.5: Live Exercise Monitoring (AI Module)

Live Exercise Monitoring (AI Module)

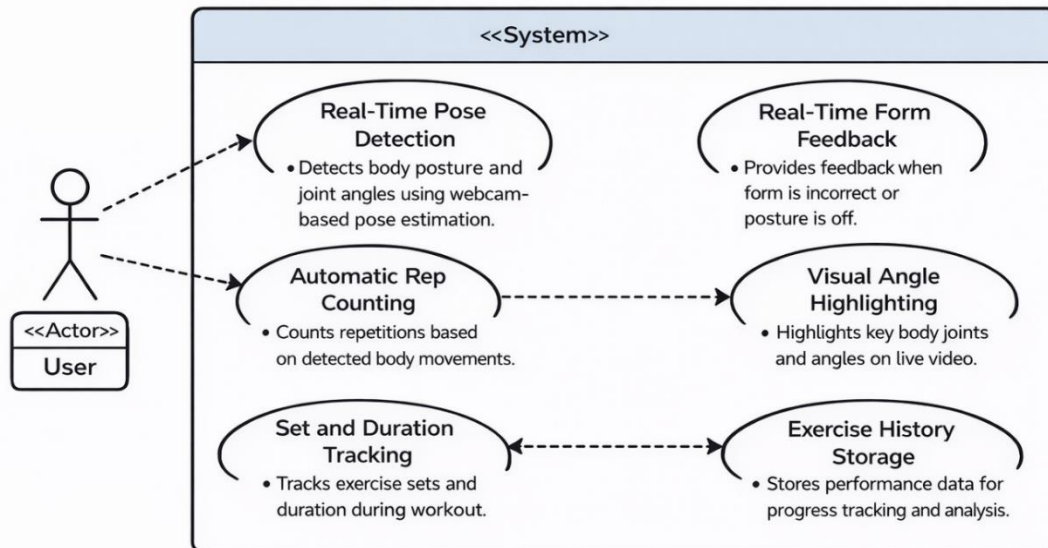


Figure 5: Live Exercise Monitoring

Req ID	Functional Requirement	Description
FR-19	Real-Time Pose Detection	The system shall detect user body posture and joint angles in real time using webcam-based pose estimation.
FR-20	Automatic Rep Counting	The system shall automatically count exercise repetitions based on detected body movements.
FR-21	Real-Time Form Feedback	The system shall provide real-time feedback when an exercise is performed incorrectly, such as incorrect angles or posture.
FR-22	Set and Duration Tracking	The system shall track exercise sets and duration during a workout session.
FR-23	Visual Angle Highlighting	The system shall visually highlight key body joints and angles (e.g., knees, elbows, shoulders) on the live video feed.
FR-24	Exercise History Storage	The system shall store exercise performance data for progress tracking and analysis.

Table 2.6: Progress Tracking

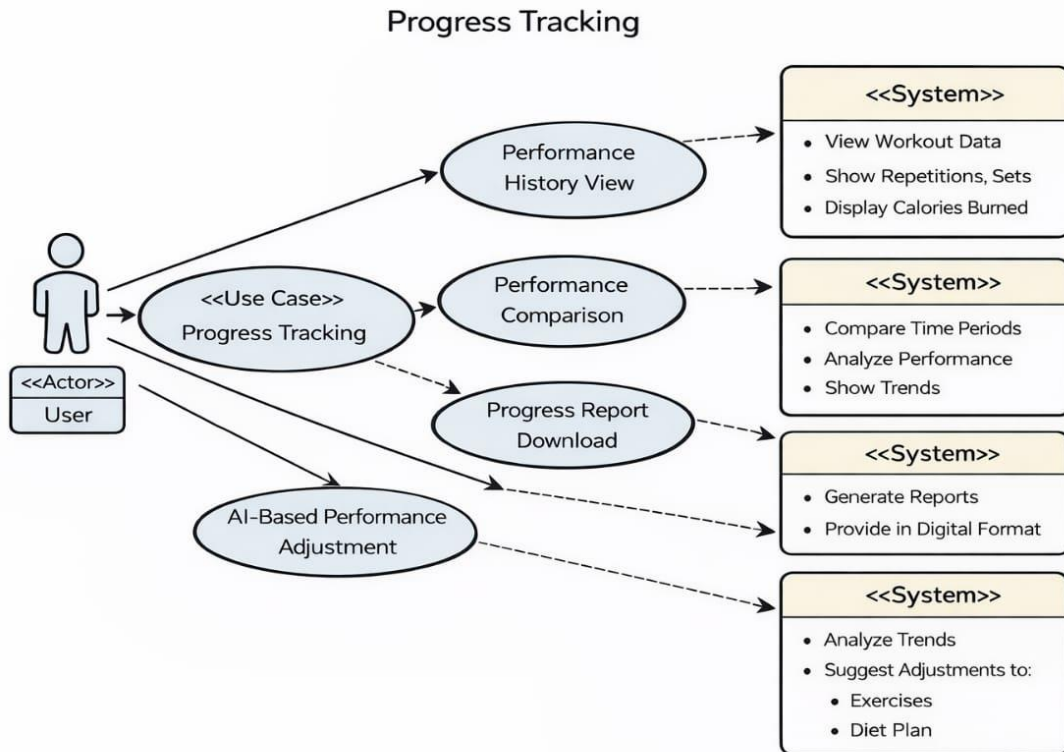


Figure 6: Progress Tracking

Req ID	Functional Requirement	Description
FR-25	Performance History View	The system shall allow users to view historical workout data, including repetitions, sets, and estimated calories burned.
FR-26	Performance Comparison	The system shall allow users to compare workout performance over different time periods such as weekly and monthly.
FR-27	Progress Report Download	The system shall allow users to download their fitness progress reports in a digital format.

FR-28	AI-Based Performance Adjustment	The system shall analyze performance trends and suggest adjustments to exercise intensity or diet plans accordingly.
-------	---------------------------------	--

Table 2.7: Notifications & Reminders

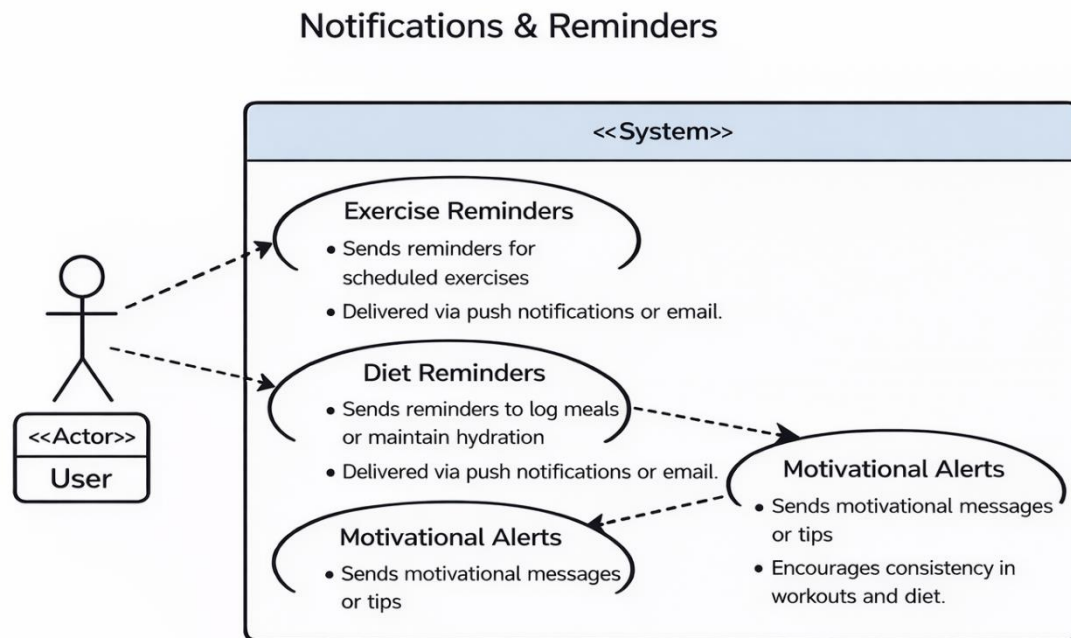


Figure 7: Notifications & Reminders

Req ID	Functional Requirement	Description
FR-29	Exercise Reminders	The system shall send reminders to users for scheduled exercises via push notifications or email.
FR-30	Diet Reminders	The system shall send reminders to users to log

		meals or maintain hydration throughout the day.
FR-31	Motivational Alerts	The system shall send motivational messages or tips to encourage users to maintain consistency in workouts and diet.

Table 2.8: Admin Functionalities

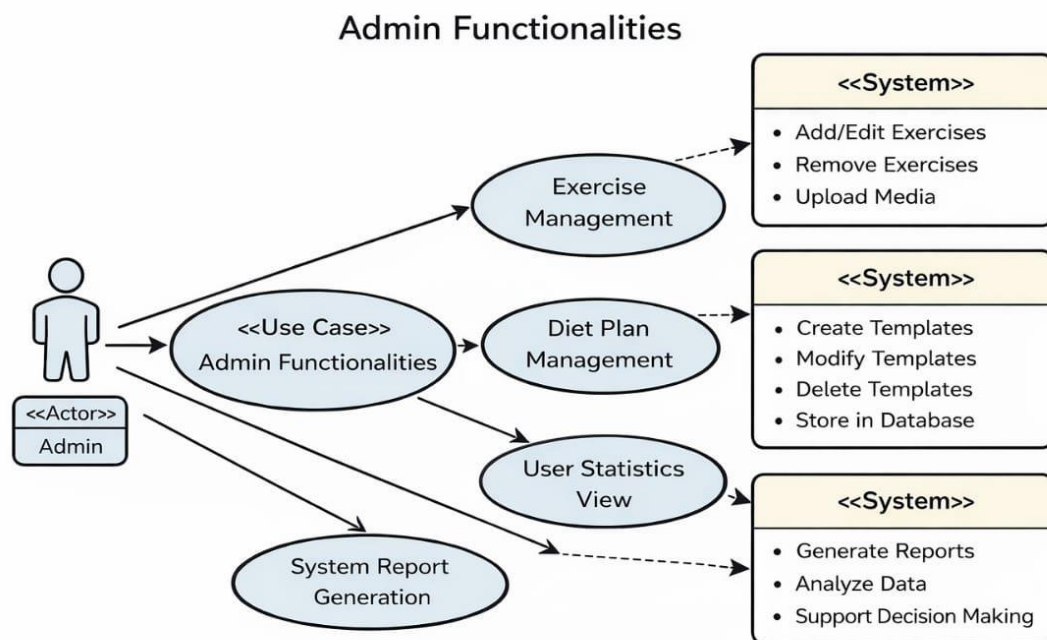


Figure 8: Admin Functionalities

Req ID	Functional Requirement	Description
FR-32	Exercise Management	The system shall allow admin users to add, remove, or edit exercises and

		associated instructions or demonstration media.
FR-33	Diet Plan Management	The system shall allow admin users to manage diet plan templates, including creation, modification, and deletion.
FR-34	User Statistics View	The system shall allow admin users to view user statistics, such as number of active users, popular exercises, and adherence trends.
FR-35	System Report Generation	The system shall allow admin users to generate system-wide reports for analysis and decision-making.

Table 2.9: Security & Data Handling

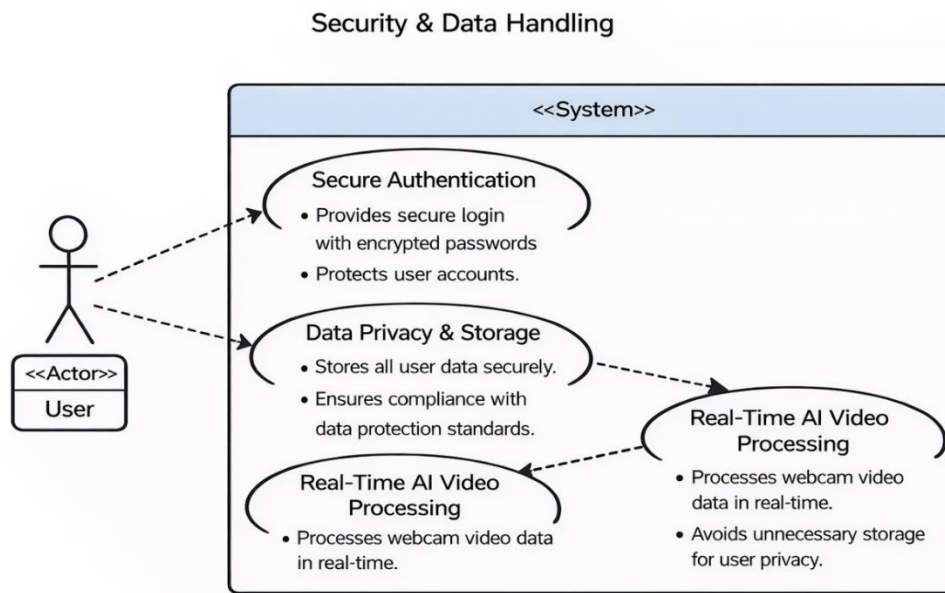


Figure 9: Security & Data Handling

Req ID	Functional Requirement	Description
--------	------------------------	-------------

FR-36	Secure Authentication	The system shall provide secure login with encrypted passwords to protect user accounts.
FR-37	Data Privacy & Storage	The system shall store all user data securely, ensuring privacy protection and compliance with relevant data protection standards.
FR-38	Real-Time AI Video Processing	The system shall process AI webcam video data in real-time and avoid unnecessary storage to ensure user privacy.

Table 2.10: Exercise Difficulty Levels

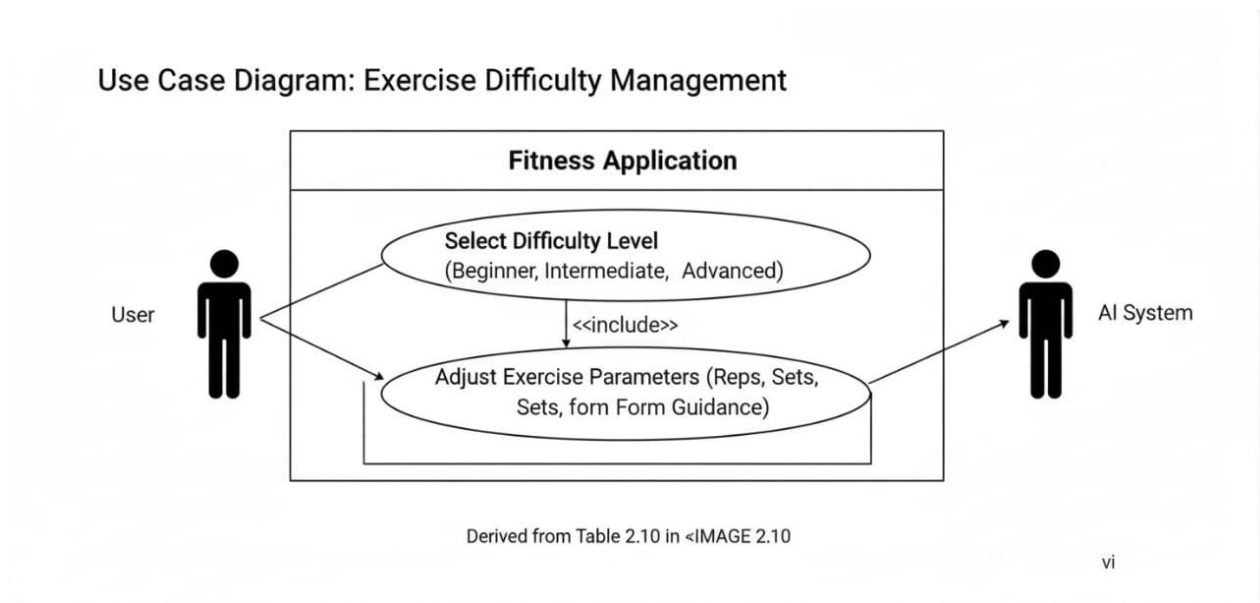


Figure 10: Exercise Difficulty levels

Req ID	Functional Requirement	Description
FR-39	Difficulty Selection	The system shall allow users to select exercise difficulty levels: beginner, intermediate, or advanced.
FR-40	AI-Based Adjustment	The system shall adjust exercise parameters such as repetitions, sets, and form guidance based on the selected difficulty level.

2.2.3. User Characteristics

The AI-Based Gym Trainer system is designed to be used by a wide range of users with varying levels of education, experience, and technical expertise. The general characteristics of the intended users are described below:

1. General Users (Fitness Enthusiasts and Beginners)

- **Educational Level:** Minimum secondary school education; no specialized technical or fitness education is required.
- **Experience:** May range from complete beginners to intermediate-level fitness enthusiasts. Users may or may not have prior experience with gym equipment or structured workout routines.
- **Technical Expertise:** Basic computer literacy, such as the ability to use a web browser, operate a webcam, and interact with a simple web interface.
- **Expectations:** Easy-to-understand guidance, real-time feedback, and simple visual indicators without complex configuration.

2. Home Workout Users

- **Educational Level:** General education background.

- **Experience:** Limited or no access to professional gym trainers; primarily rely on online videos or self-guided workouts.
- **Technical Expertise:** Basic to moderate technical knowledge, sufficient to install or access a web-based application.
- **Expectations:** Affordable, accessible, and reliable virtual training assistance with minimal setup requirements.

3. Fitness-Conscious Individuals

- **Educational Level:** Secondary or higher education.
- **Experience:** Regular gym-goers or individuals focused on maintaining proper exercise form and performance.
- **Technical Expertise:** Comfortable using modern web applications and basic digital tools.
- **Expectations:** Accurate posture analysis, precise repetition counting, and meaningful feedback to enhance workout quality.

4. System Administrators / Developers (Secondary Users)

- **Educational Level:** Undergraduate level or higher in Computer Science, Software Engineering, or a related field.
- **Experience:** Experience in web development, machine learning, or system maintenance.
- **Technical Expertise:** Advanced technical knowledge, including system configuration, model updates, debugging, and performance optimization.
- **Responsibilities:** Managing system updates, maintaining databases, improving models, and ensuring system reliability.

2.2.4 Constraints

This subsection describes the constraints that may limit the design, development, implementation, and deployment options of the AI-Based Gym Trainer system. These constraints must be considered throughout the software development lifecycle.

1. Regulatory Policies

- The system must comply with general data protection and privacy policies related to user information.
- User data such as login credentials and exercise history must be stored securely and used only for system functionality.
- The application does not provide medical or clinical advice and must include a disclaimer to avoid regulatory issues related to healthcare systems.

2. Hardware Limitations

- The system relies on the availability and quality of a webcam; low-resolution cameras may affect pose detection accuracy.
- Performance may vary depending on the user's device specifications, such as CPU speed, GPU availability, and RAM.
- Real-time processing may be limited on low-end or older hardware systems.

3. Interfaces to Other Applications

- The system may interface with web browsers, databases, and optional cloud services.
- Integration with third-party fitness applications or wearable devices is not supported in the current version.
- External libraries such as MediaPipe and OpenCV impose their own compatibility and version constraints.

4. Parallel Operation

- The system is designed primarily for single-user operation per session.
- Running multiple instances of real-time pose detection on the same device may degrade system performance.

- Parallel execution of other resource-intensive applications may affect real-time feedback accuracy.

5. Audit Functions

- Basic logging of user sessions, exercise performance, and system errors may be maintained.
- Advanced auditing and compliance reporting features are not included in the current scope.
- Logs are primarily intended for debugging, performance monitoring, and future system improvements.

6. Control Functions

- The system operates based on predefined exercise rules, angle thresholds, and movement patterns.
- Users have limited control over modifying internal pose detection parameters.
- Administrative control is required for updating models, thresholds, and system configurations.

7. Higher-Order Language Requirements

- The core system must be developed using Python for machine learning and computer vision tasks.
- JavaScript is required for frontend web development and user interaction.
- Compatibility with supported programming languages and frameworks is mandatory for system integration.

8. Signal Handshake Protocols

- Communication between frontend and backend modules must follow standard web communication protocols (HTTP/HTTPS).
- Real-time video streaming and feedback signals must be synchronized to ensure accurate posture evaluation.
- Audio feedback signals must be triggered promptly upon detection of incorrect posture.

9. Reliability Requirements

- The system must operate reliably during exercise sessions without frequent crashes or delays.
- Temporary failures such as webcam disconnection should be handled gracefully with appropriate user notifications.
- The system should maintain consistent performance under normal usage conditions.

10. Criticality of the Application

- The system is classified as a **non-life-critical application**.
- Incorrect feedback may affect workout quality but does not pose immediate life-threatening risks.
- Users are advised to exercise caution and consult professionals for medical concerns.

11. Safety and Security Considerations

- User authentication must be implemented to prevent unauthorized access.
- Data transmission should be secured using encryption where applicable.
- The system must ensure that no harmful or misleading instructions are given during exercise.
- Users should be warned to stop exercising if discomfort or pain occurs.

2.2.5 Assumptions and Dependencies

This subsection outlines the assumptions and dependencies that may influence the requirements and behaviour of the AI-Based Gym Trainer system. These factors are external to the software design but can directly impact system performance, functionality, and reliability if they change.

Assumptions

1. Availability of Hardware

- It is assumed that users have access to a functional desktop or laptop with a built-in or external webcam.
- Audio output devices (speakers or headphones) are available for receiving voice or buzzer feedback.

2. User Environment

- Users will perform exercises in a well-lit environment to ensure accurate pose detection.
- The camera will be positioned properly to capture the full body or required body parts during exercises.
- The user will face the camera as recommended by the system.

3. User Behaviour

- Users will follow on-screen instructions and system guidelines while performing exercises.
- Users will select exercises that are supported by the system.
- Users will not rely on the system for medical diagnosis or treatment.

4. Internet Connectivity

- A stable internet connection is assumed for user authentication, data storage, and accessing web-based features.
- Temporary offline operation may be limited or unavailable depending on implementation.

5. Exercise Dataset Accuracy

- The predefined exercise rules, angle thresholds, and posture standards are assumed to be correctly defined and validated.

- The system assumes that supported exercises follow standard and commonly accepted workout forms.

Dependencies

1. Third-Party Libraries and Frameworks

- The system depends on external libraries such as Media Pipe and OpenCV for pose detection and video processing.
- Any changes, deprecations, or updates in these libraries may affect system functionality.

2. Web Browser Compatibility

- The system depends on modern web browsers that support webcam access and real-time processing.
- Browser security policies and updates may affect webcam and audio permissions.

3. Operating System Support

- The application's performance may depend on the underlying operating system (Windows, macOS, or Linux).
- OS-level updates may impact hardware drivers such as webcam or audio devices.

4. Backend and Database Services

- User account management and exercise history storage depend on the availability of backend servers and database systems.
- Server downtime or database failures may temporarily affect system availability.

5. Machine Learning Model Performance

- The accuracy of posture detection and repetition counting depends on the trained machine learning models.

- Model performance may vary across different body types, clothing, and movement styles.

6. Deployment Environment

- The system depends on proper deployment configuration, including server setup and application hosting.
- Changes in hosting services or deployment platforms may require system adjustments.

2.2.6 Apportioning of Requirements

This subsection identifies the system requirements that may not be implemented in the initial version of the AI-Based Gym Trainer but can be included in future releases. Apportioning requirements helps prioritize core functionalities while allowing for future enhancements.

Requirements Delayed for Future Versions

1. Support for Additional Exercises

- The initial version will include a limited set of predefined exercises (e.g., squats, push-ups, and bicep curls).
- Support for a wider range of exercises, such as lunges, deadlifts, or advanced yoga poses, will be added in future versions.

2. Integration with Wearable Devices

- Real-time integration with smartwatches, fitness bands, or heart rate monitors will be considered in future updates.
- This will allow more precise tracking of physiological metrics alongside posture analysis.

3. Cloud-Based Data Storage and Synchronization

- Currently, user data may be stored locally or on a simple backend server.

- Full cloud-based storage, synchronization across devices, and backup functionality can be implemented in future versions.

4. Advanced Analytics and Progress Reports

- Basic repetition counting and posture correction will be available initially.
- Detailed analytics, performance trends, exercise difficulty suggestions, and graphical progress reports may be introduced in future releases.

5. AI Model Customization

- The first version will use general pose estimation models trained for average body types.
- Future versions could include personalized AI models trained for individual body types, fitness levels, or custom exercise routines.

6. Multi-User and Group Session Support

- The system will initially support single-user operation per session.
- Features for monitoring multiple users simultaneously or hosting virtual group sessions may be developed later.

7. Mobile Application Support

- The first release will focus on web-based access.
- A fully functional mobile application for Android and iOS may be released in future versions.

8. Voice Assistant Integration

- Basic voice commands and alerts will be implemented initially.
- Integration with advanced voice assistants like Alexa or Google Assistant for personalized workout guidance can be included in future updates.

2.3 Specific Requirements

This section defines the detailed requirements of the AI-Based Gym Trainer system, including both functional and non-functional requirements. These requirements provide a clear blueprint for designers and developers to build, test, and deploy the system according to user needs.

2.3.1 Functional Requirements

The functional requirements define the specific functions or modules that the system must perform to fulfill user objectives. These modules are as follows:

1. User Account Management

- Create, edit, and delete user accounts.
- Secure login and logout functionality.
- Store user profile information including name, email, body type, and exercise history.

2. Exercise Selection Module

- Allows users to select predefined exercises from a list.
- Customize exercise plans based on user body type or fitness goals.

3. Webcam-Based Video Capture

- Capture live video feed of the user performing exercises.
- Continuously stream frames to the pose detection module.

4. Human Pose Detection Module

- Detect key body landmarks using machine learning models (MediaPipe).
- Identify critical joints such as elbows, knees, shoulders, hips, and ankles.

5. Joint Angle Calculation Module

- Compute angles between detected joints in real-time.
- Compare calculated angles against predefined correct ranges for exercise evaluation.

6. Exercise Repetition Counting

- Track the number of correctly performed repetitions.
- Reset and update count dynamically during exercise sessions.

7. Incorrect Posture Detection

- Detect improper movements or deviations from correct posture.
- Flag incorrect posture and provide immediate feedback.

8. Real-Time Feedback Module

- Provide voice or buzzer alerts when incorrect posture is detected.
- Give visual indicators on the screen for better understanding.

9. Exercise Session Monitoring

- Display live video feed with posture evaluation overlays.
- Show real-time repetition count and session progress.

10. Data Storage and History Management

- Store exercise session data locally or on the backend.
- Maintain user history for progress tracking and analytics.

2.3.2 Non-Functional Requirements

The non-functional requirements define the quality attributes and operational constraints of the system, including usability, reliability, performance, and maintainability.

1. Usability

- The system must have an intuitive web-based interface that is easy to navigate.
- Minimal user interaction should be required during exercise sessions.
- Visual cues and alerts must be clear and understandable for all user types.

2. Reliability

- The system must provide consistent real-time pose detection and repetition counting.
- Temporary errors such as webcam disconnection must be handled gracefully with notifications.
- High availability during exercise sessions is required.

3. Performance

- Pose detection and feedback generation must occur with minimal delay (<1 second).
- The system should maintain smooth video processing without frame drops on standard hardware.

4. Design Constraints

- The system must be implemented using Python for machine learning modules and JavaScript for web application interface.
- MediaPipe and OpenCV must be used for pose detection and video processing.
- Web-based platform must be compatible with modern browsers supporting webcam access.

5. Portability

- The system should run on multiple operating systems including Windows, macOS, and Linux.
- Web-based design ensures accessibility from different devices with internet access.

6. Maintainability

- Modular code structure to facilitate easy updates and debugging.
- Proper documentation of functions, classes, and modules for future developers.
- Model updates and threshold modifications should be straightforward to implement.

7. License Agreement

- The system will use open-source libraries such as MediaPipe and OpenCV, adhering to their respective licenses.
- Any redistribution of the system must comply with open-source license terms and proper attribution.

List of Tables:

Table 1: Functional Requirements of the AI-Based Gym Trainer System

ID	Functional Requirement	Description	Inputs	Outputs	Requirements	Basic Workflow
FR_01	Create Account	Allow users to create a new account.	Name, Email, Password	Account Created	Internet connectivity	User enters details and submits the form.
FR_02	Login Account	User logs into the system.	Email, Password	Access Granted	Valid credentials	User enters email and password.
FR_03	Logout Account	User logs out of the system.	None	Logged Out	None	User clicks on logout button.
FR_04	Perform Exercise	Tracks user's exercise via webcam.	Webcam Feed	Reps Counted, Feedback Given	Webcam, MediaPipe, OpenCV	System analyzes exercise and gives feedback.
FR_05	Select Exercise	Choose an exercise to perform.	Exercise List	Exercise Selected	None	User selects an exercise.
FR_06	Select Body Type	Select body type: Muscular, Bulky, Slim Fit.	Body Type Option	Body Type Chosen	None	User selects desired body type.
FR_07	Select Diet Plan	Choose a diet plan.	Diet Plan Option	Diet Plan Selected	None	User selects a diet plan.
FR_08	View Progress	View user's workout progress.	None	Progress Displayed	Previous Data	System shows progress report.
FR_09	Update Profile	Edit user profile details.	Profile Details	Profile Updated	Internet connectivity	User updates profile information.
FR_10	Receive Alerts	Get alerts for incorrect exercises.	Incorrect Exercise	Alert / Voice / Buzzer	Webcam, MediaPipe, OpenCV	System alerts for wrong exercise.
FR_11	View Tips	Display fitness tips and advice.	None	Tips Displayed	None	User views tips and advice.

Table 2: Non-Functional Requirements of the AI-Based Gym Trainer System

ID	Non-Functional Requirement	Description
NFR_01	Performance	The system should process webcam frames and calculate exercise angles in real time without noticeable delay.
NFR_02	Accuracy	The system should accurately detect body posture, joint angles, and count exercise repetitions correctly.
NFR_03	Usability	The system should be easy to use for users with no technical background.
NFR_04	Reliability	The system should work continuously without crashes or incorrect behavior during exercise sessions.
NFR_05	Scalability	The system should support future expansion, such as adding new exercises or features.
NFR_06	Security	User data and exercise records should be securely stored and protected from unauthorized access.
NFR_07	Compatibility	The system should work on web-based platforms and desktop applications using modern browsers.
NFR_08	Response Time	The system should provide instant feedback through voice commands or a buzzer when an incorrect exercise is detected.
NFR_09	Maintainability	The system code should be modular and easy to update or maintain in the future.
NFR_10	Availability	The system should be available whenever the user wants to perform an exercise session.

Table 3: Use Case Description – Exercise Monitoring and Feedback

Item	Description
Use Case ID	UC_01
Use Case Name	Exercise Monitoring and Feedback
Actor	User
Description	The system monitors the user's exercise through a webcam, calculates joint angles, counts repetitions, and provides real-time feedback if the exercise is performed incorrectly.
Pre-Conditions	User is logged into the system and webcam is enabled.
Post-Conditions	Exercise data is recorded and feedback is provided to the user.
Input	Live webcam video feed of the user performing exercises.
Output	Exercise repetition count, angle analysis, and real-time feedback via voice command or buzzer.
Main Flow	1. User selects an exercise. 2. User starts performing the exercise in front of the webcam. 3. System detects body landmarks using MediaPipe. 4. Joint angles are calculated using OpenCV. 5. Repetitions are counted. 6. System provides feedback if the exercise is incorrect.
Alternate Flow	If incorrect posture or movement is detected, the system alerts the user through a voice command or buzzer.
Exceptions	Webcam not detected or poor lighting conditions may affect detection accuracy.
Non-Functional Requirements	Real-time performance, high accuracy, usability, and reliability.
Tools & Technologies	Python, MediaPipe, OpenCV, Machine Learning, JavaScript (for web/application interface).

Table 4: Input and Output Specifications

Input ID	Input Description	Source	Output ID	Output Description	Destination
IN_01	User Registration Details (Name, Email, Password)	User	OUT_01	Account Creation Confirmation	User
IN_02	Login Credentials (Email, Password)	User	OUT_02	Login Access Granted / Denied	User
IN_03	Live Webcam Video Feed	Webcam	OUT_03	Body Pose Landmarks	Pose Detection Module
IN_04	Body Joint Coordinates	MediaPipe	OUT_04	Calculated Joint Angles	Angle Calculation Module
IN_05	Exercise Movement Data	OpenCV	OUT_05	Exercise Repetition Count	User Interface
IN_06	Incorrect Exercise Detection	ML Model	OUT_06	Voice Feedback / Buzzer Alert	User
IN_07	Exercise Selection	User	OUT_07	Selected Exercise Loaded	System
IN_08	User Workout Data	System Database	OUT_08	Progress Report	User
IN_09	Profile Update Information	User	OUT_09	Profile Updated Confirmation	User

Table 5: Tools, Libraries, and Technologies Used

Sr. No.	Tool / Library / Technology	Purpose / Usage
1	Python	Core programming language for machine learning model development and backend logic.
2	MediaPipe	Used for real-time human pose detection and body landmark extraction.
3	OpenCV	Used for video capture, image processing, and angle calculation.
4	Machine Learning	Used to analyze exercise patterns and detect correct or incorrect posture.
5	JavaScript	Used for developing the web-based application interface.
6	HTML	Used to design the structure of the web application.
7	CSS	Used for styling and layout of the web application.
8	Flask / Django (Optional)	Used to connect the Python backend with the web application.
9	Web Camera	Used to capture live video of the user during exercise.
10	Voice/Buzzer Module	Used to provide audio alerts when an incorrect exercise is detected.
11	Visual Studio Code	Integrated Development Environment (IDE) for coding and debugging.
12	Git / GitHub	Used for version control and project management.

Table 6: Hardware and Software Requirements

Category	Requirement	Specification
Hardware	Processor	Intel Core i3 or higher
Hardware	RAM	Minimum 8 GB
Hardware	Storage	Minimum 128 GB HDD/SSD
Hardware	Webcam	HD Webcam (570p or higher)
Hardware	Audio Output	Speaker or Buzzer Module
Hardware	GPU (Optional)	NVIDIA GPU for faster processing
Software	Operating System	Windows 10 / Linux / macOS
Software	Programming Language	Python 3.x
Software	Libraries	MediaPipe, OpenCV, NumPy
Software	Web Technologies	HTML, CSS, JavaScript
Software	Backend Framework	Flask or Django
Software	IDE	Visual Studio Code, Jupyter

Table 7: Supported Exercises and Joint Angle Thresholds

Exercise ID	Exercise Name	Primary Joints Monitored	Angle Threshold (Correct Form)	Description
EX_01	Bicep Curl	Elbow	30° – 160°	Elbow should fully extend and flex within the defined angle range during repetitions.
EX_02	Squats	Knee, Hip	Knee: 70° – 160°	Knees and hips should bend properly while maintaining correct posture.
EX_03	Push-Ups	Elbow, Shoulder	Elbow: 40° – 160°	Elbows should bend and extend correctly while keeping the body straight.
EX_04	Shoulder Press	Shoulder, Elbow	Shoulder: 60° – 180°	Arms should move upward symmetrically with controlled shoulder motion.
EX_05	Lunges	Knee, Hip	Knee: 70° – 150°	Front knee should bend properly without crossing the toe line.
EX_06	Plank	Shoulder, Hip	Shoulder: 160° – 180°	Body should remain straight with minimal joint angle variation.

EX_07	Jumping Jacks	Shoulder, Hip	Shoulder: 60° – 180°	Arms and legs should move in synchronized motion.
-------	---------------	---------------	----------------------	---

Table 8: Dataset Description and Pose Features

Feature ID	Feature Name	Description	Source	Purpose
DF_01	Video Frames	Individual frames extracted from live webcam feed.	Webcam	Used for real-time pose detection.
DF_02	Body Landmarks	33 human body key points (x, y, z coordinates).	MediaPipe Pose	Used to identify body posture and movement.
DF_03	Joint Angles	Angles between connected joints (e.g., elbow, knee).	OpenCV Calculations	Used to evaluate correct or incorrect exercise form.
DF_04	Repetition Count	Number of completed exercise cycles.	ML Logic	Used to track exercise repetitions.
DF_05	Posture Label	Label indicating correct or incorrect posture.	ML Model	Used for supervised learning and feedback generation.
DF_06	Exercise Type	Type of exercise being performed.	User Selection	Used to apply exercise-specific rules and thresholds.
DF_07	Movement Speed	Speed of joint movements during exercise.	Frame Analysis	Used to analyze exercise consistency and control.
DF_08	Angle Threshold Values	Predefined acceptable angle ranges.	System Configuration	Used to validate exercise form.
DF_09	Feedback Trigger	Condition indicating incorrect movement.	Rule-Based Logic	Used to activate voice or buzzer alerts.
DF_10	Timestamp	Time at which each frame or movement is captured.	System Clock	Used for performance analysis and progress tracking.

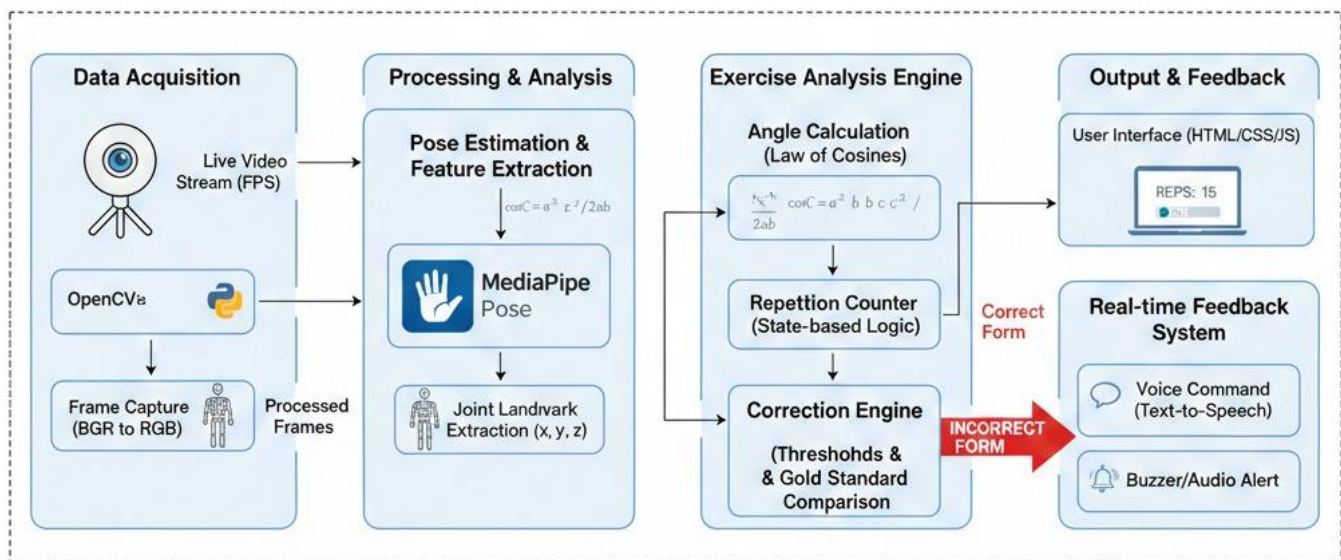
Table 9: Model Training and Performance Evaluation Metrics

Metric ID	Metric Name	Description	Calculation / Formula	Purpose
MET_01	Accuracy	Measures how correctly the model detects correct and incorrect exercises.	$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$	To evaluate overall correctness of the model predictions.
MET_02	Precision	Measures proportion of correctly predicted incorrect exercises.	$\text{Precision} = \frac{TP}{TP + FP}$	To ensure the model does not give false alerts.
MET_03	Recall (Sensitivity)	Measures proportion of actual incorrect exercises detected.	$\text{Recall} = \frac{TP}{TP + FN}$	To ensure incorrect exercises are not missed.
MET_04	F1 Score	Harmonic mean of precision and recall.	$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$	To balance precision and recall in model evaluation.
MET_05	Repetition Counting Accuracy	Measures how accurately the model counts exercise repetitions.	$\text{Reps Accuracy} = \frac{\text{Correctly Counted Reps}}{\text{Total Reps}} \times 100\%$	To track exercise progress reliably.
MET_06	Mean Absolute Error (MAE)	Measures average error in predicted joint angles vs. actual angles.	$\text{MAE} = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	
MET_07	Real-Time Performance	Measures processing speed of model (frames per second).	Frames processed / second	To ensure system provides real-time feedback.
MET_08	Model Loss	Loss function value during training (e.g., cross-entropy).	Computed by model	To monitor training progress and avoid overfitting.
MET_09	Confusion Matrix	Summarizes True Positives, True Negatives, False Positives, False Negatives.	Tabular calculation	To analyze types of prediction errors.
MET_10	System Response Time	Time taken from detecting incorrect posture to providing feedback.	Measured in milliseconds	To ensure immediate feedback for user correction.

Chapter 4: Design

4.1 Architecture Diagram:

GYM TRAINER ARCHITECTURE DIAGRAM

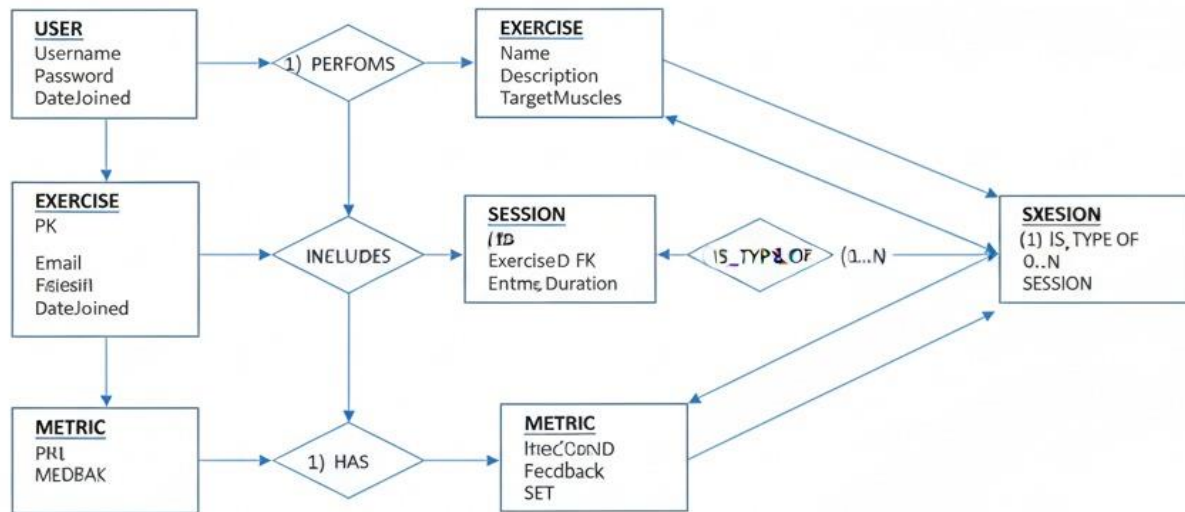


BSCS FINAL YEAR PROJECT

Figure 11: Architecture Diagram

4.2 ERD with data dictionary

GYM TRAINER - ENTITY-RELATONSHIP DIAGRAM (ERD)



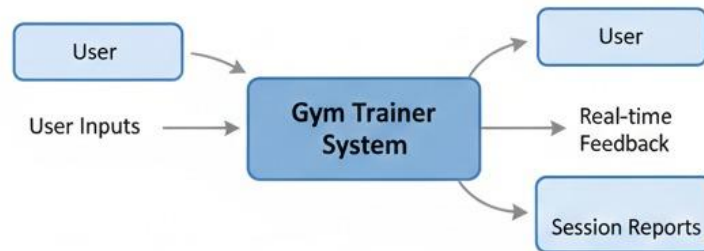
DATA DICTIONARY

Entity/Attribute	Description	Data Type	Constraints/Notes
USER	Unique identifier for user	Integer	PK, Auto-increment
Name	User's chosen name	Varchar(50)	PK to USER, or iUIDic)
Name	User's identifier for an session.	Varchar(100)	PK to USER(255)
ExerciseID	User who performed (e.g., "Bicep Curl)	Integer	Varchar(100)
ExerciseID	Description	Integer	FK to EXERCISE.SSID)
ExerciseID	Exercise performed	Integer	FK to SESSION.SESSID)
RepCount	Session the set bellmsted	Not Null	Float(255)
Feedback	Session the set completed	Not Null	"Keep cep elbow in")
TimeSpent	Total duration in in second)	Not Null	Date/Text
Metric	Text feedback (e. gl. "Great angles over time")	Not Null	JSON/Text
RepQualityScore	Calculated metrics score (0-100)	Not Null	Not Null

Figure 12: ERD with data Dictionary

4.3 Data Flow diagram

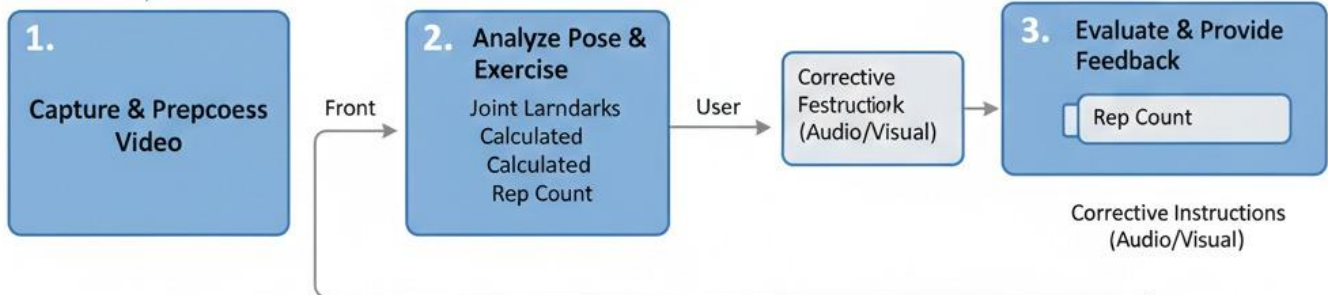
GYM TRAINER - DATA FLOW DIAGRAM (DFF)



Level 0: Context Diagram



Level 1: System Decomposition



Level 4: Manage User Sessions & Reports

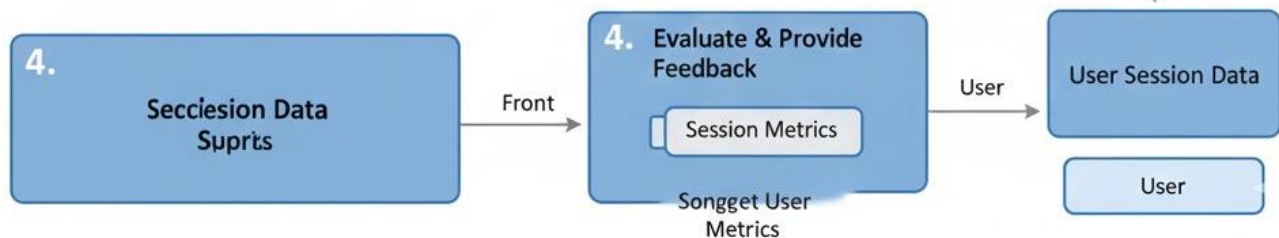


Figure 13: Data Flow Diagram

4.4 Class Diagram

GYM TRAINER - CLASS DIAGRAM

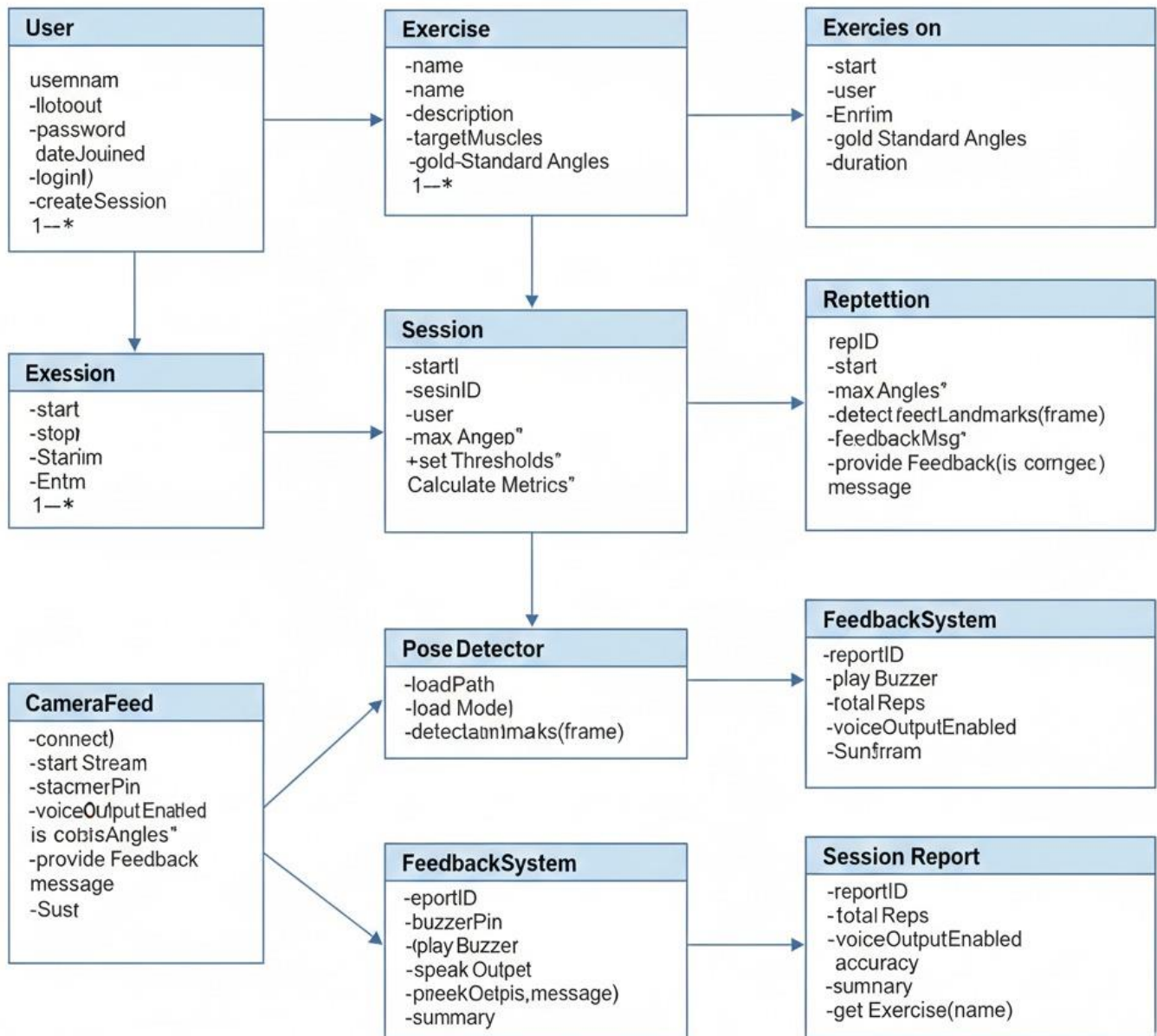


Figure 14: Class Diagram

4.5 Activity Diagram

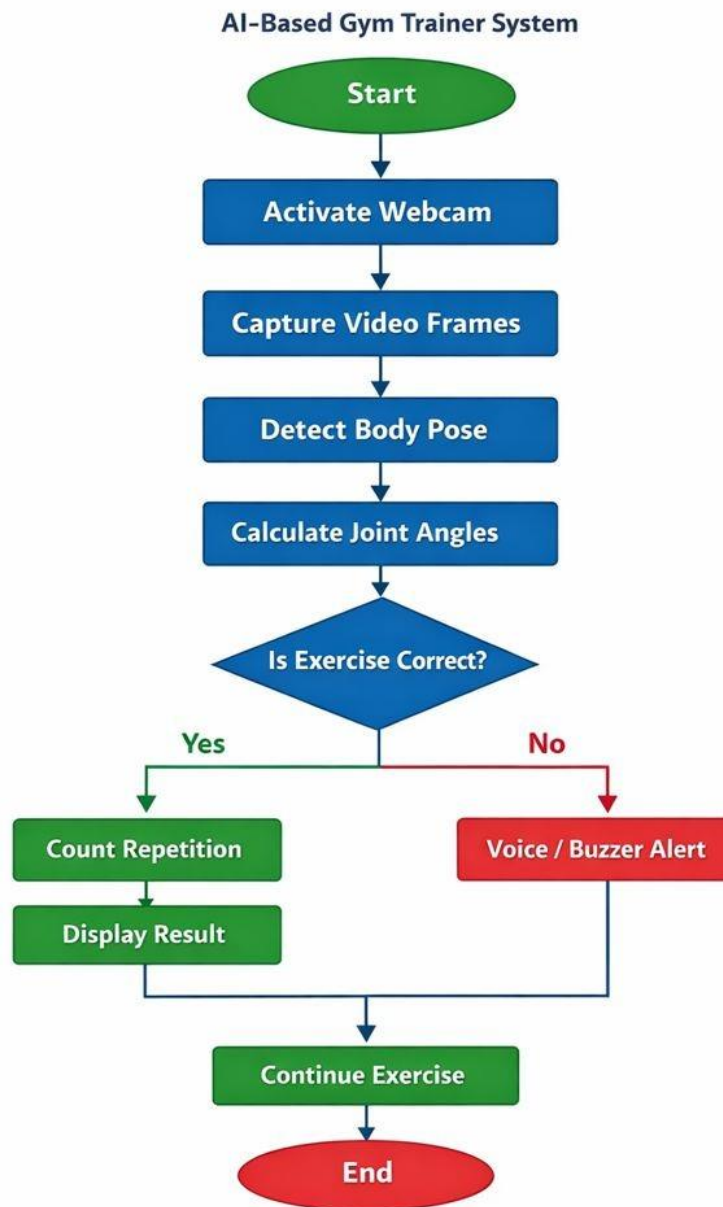


Figure 15: Activity Diagram

4.6 Sequence Diagram

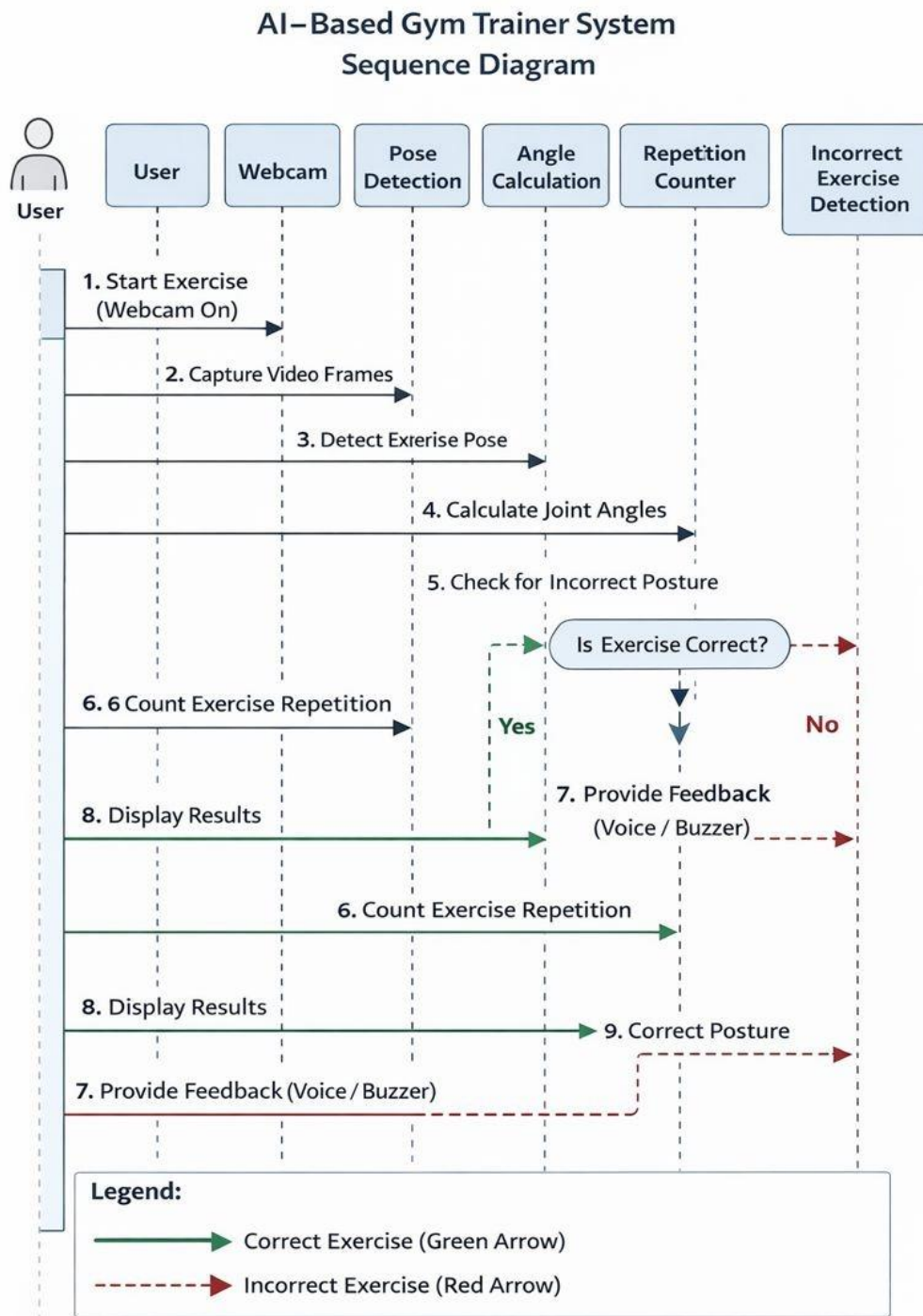


Figure 16: Sequence Diagram

4.7 Collaboration Diagram

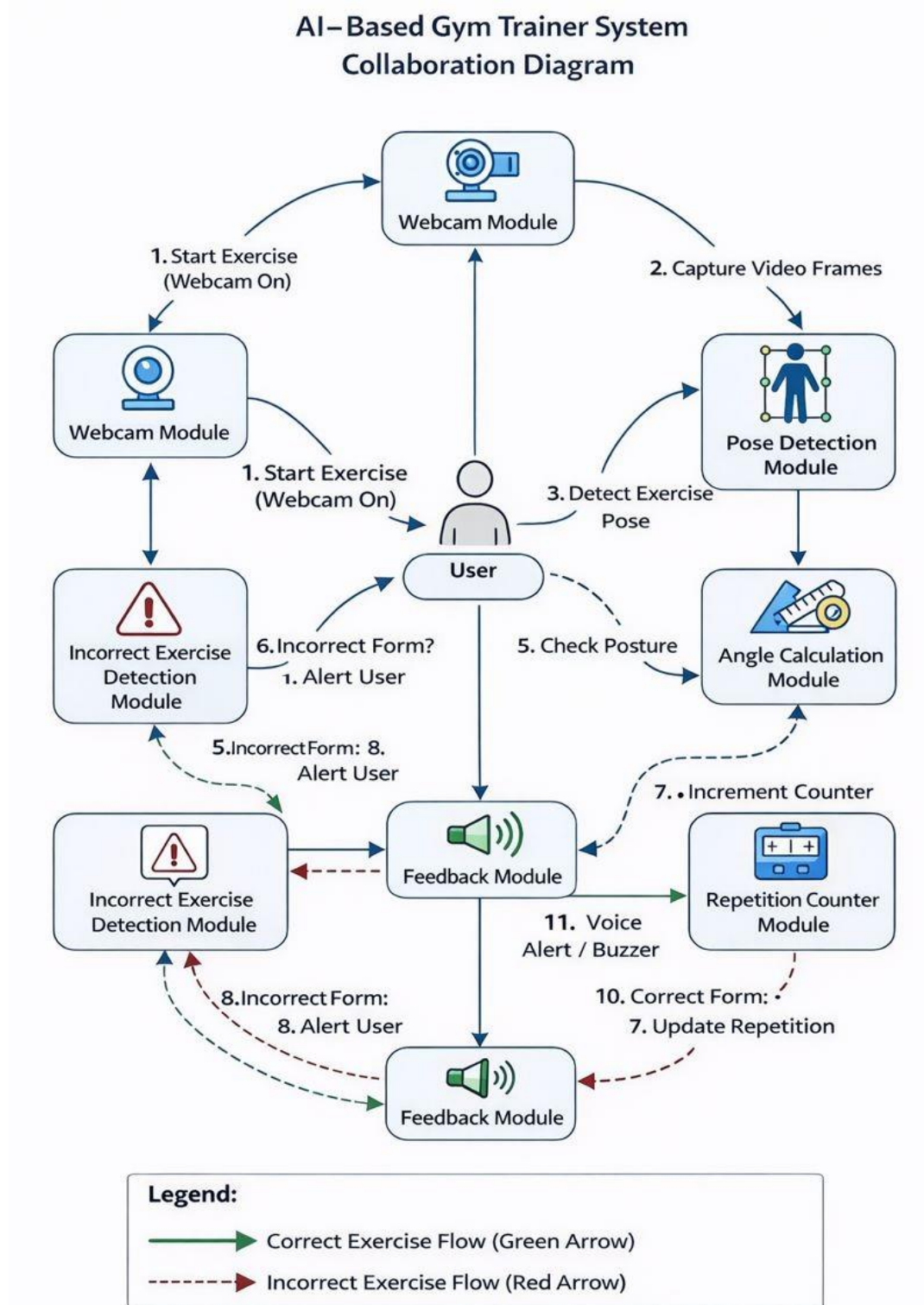


Figure 17: Collaboration Diagram

4.8 State Transition Diagram

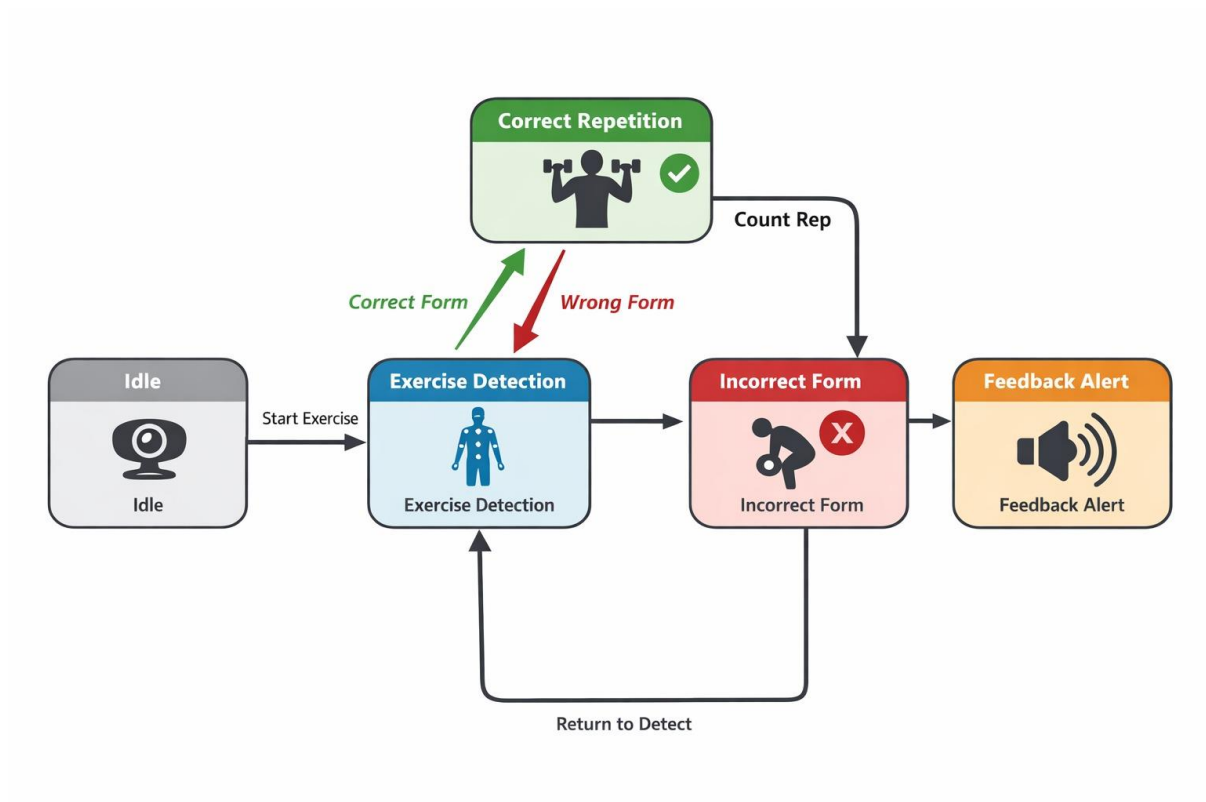


Figure 18: State Transition Diagram

4.9 Component Diagram

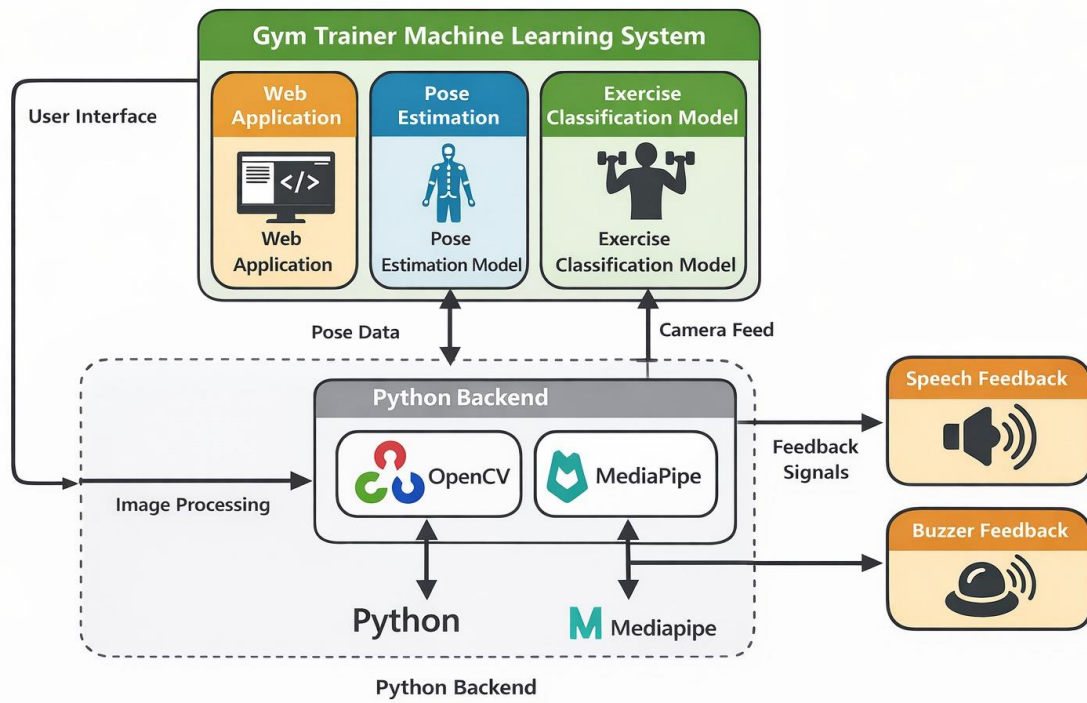


Figure 19: Component Diagram

4.10 Deployment Diagram:

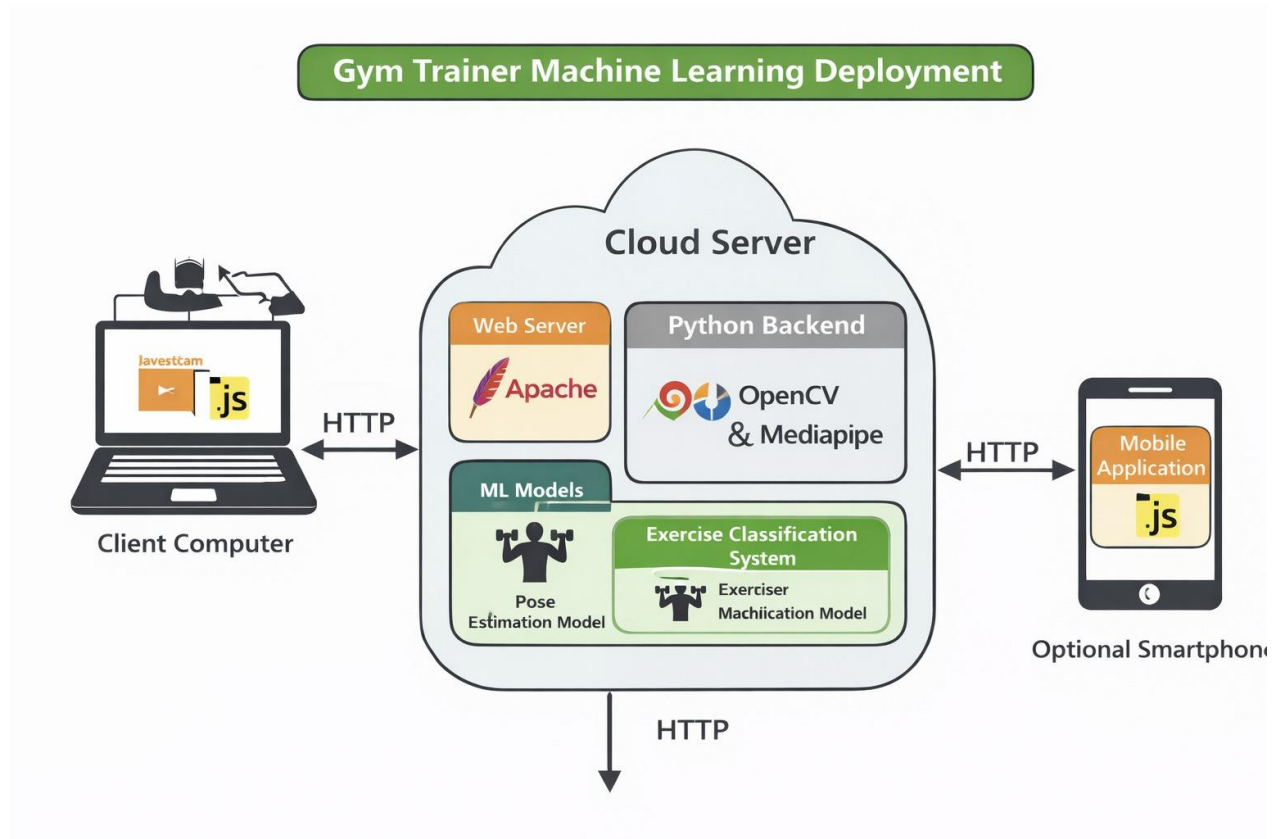


Figure 20: Deployment Diagram

Chapter 5: User Manual

The User Manual provides a detailed guideline for operating the **AI-Based Gym Trainer system** developed so far. The current version of the application focuses on analyzing **bicep curl exercises** using a webcam by detecting body landmarks, measuring joint angles, and counting exercise repetitions. To use the system, the user opens the web-based application and allows access to the webcam. After camera activation, the user positions themselves in front of the camera so that the upper body and arms are clearly visible. The system employs **MediaPipe** for real-time human pose detection and **OpenCV** for processing video frames. As the user performs bicep curl movements, the application tracks the upward and downward motion of the arm, calculates elbow joint angles by drawing landmark-based lines, and counts repetitions based on predefined angle ranges. At this stage, the system displays real-time visual feedback including detected body landmarks, joint angle values, and repetition count on the screen. Detection of incorrect exercise posture and automated corrective feedback through voice or buzzer alerts is **not yet implemented** and is planned as a future enhancement. Screenshots of the frontend interface are included in this chapter to illustrate the working flow of the application, including camera initialization, pose detection, angle visualization, and repetition counting.

Images of the Model:

```
[94]: import cv2

# Initialize webcam (0 = default camera)
cap = cv2.VideoCapture(0)

# Check if camera opened successfully
if not cap.isOpened():
    print("✗ Error: Could not open webcam. Please check connection or permissions.")
else:
    print("✓ Webcam opened successfully. Press 'q' to quit.")

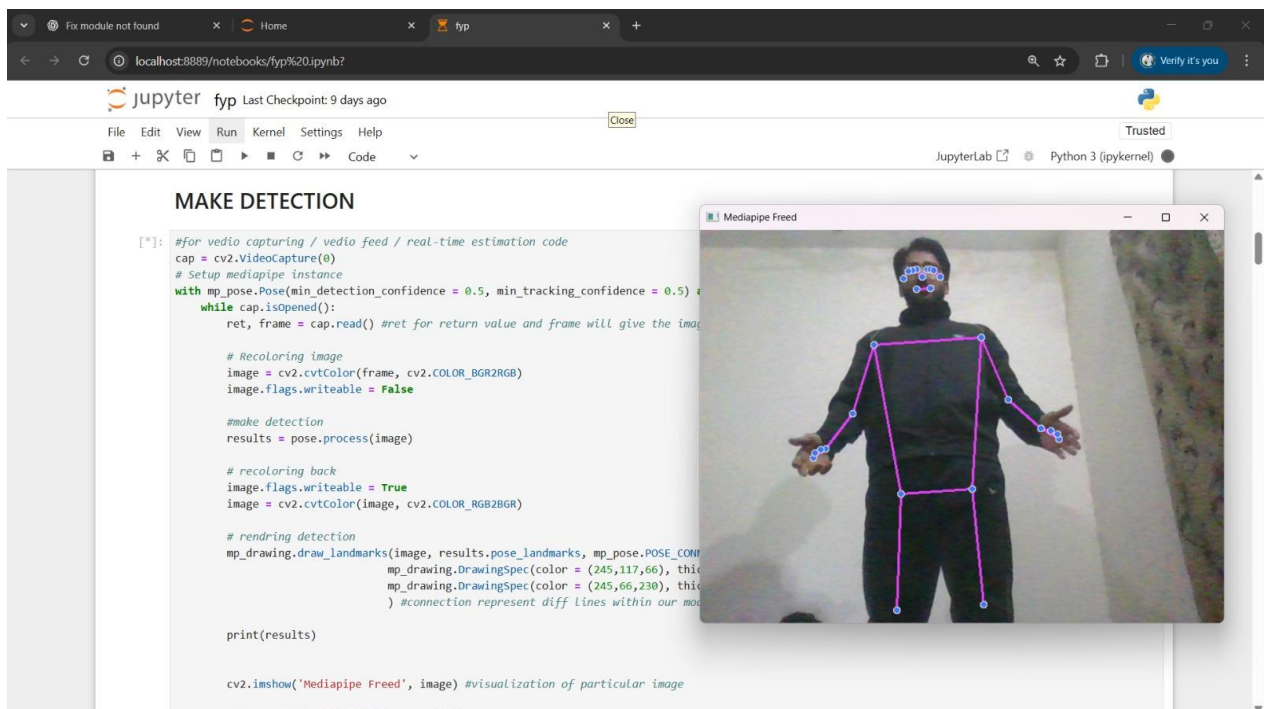
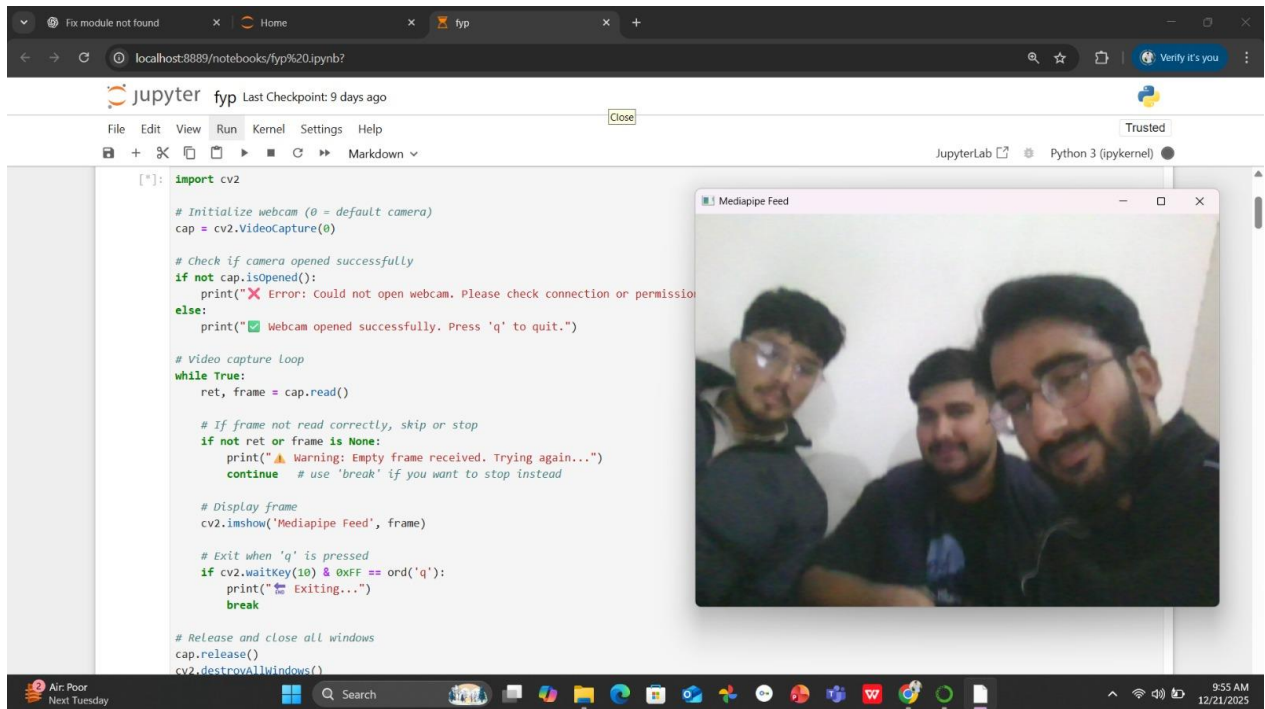
# Video capture loop
while True:
    ret, frame = cap.read()

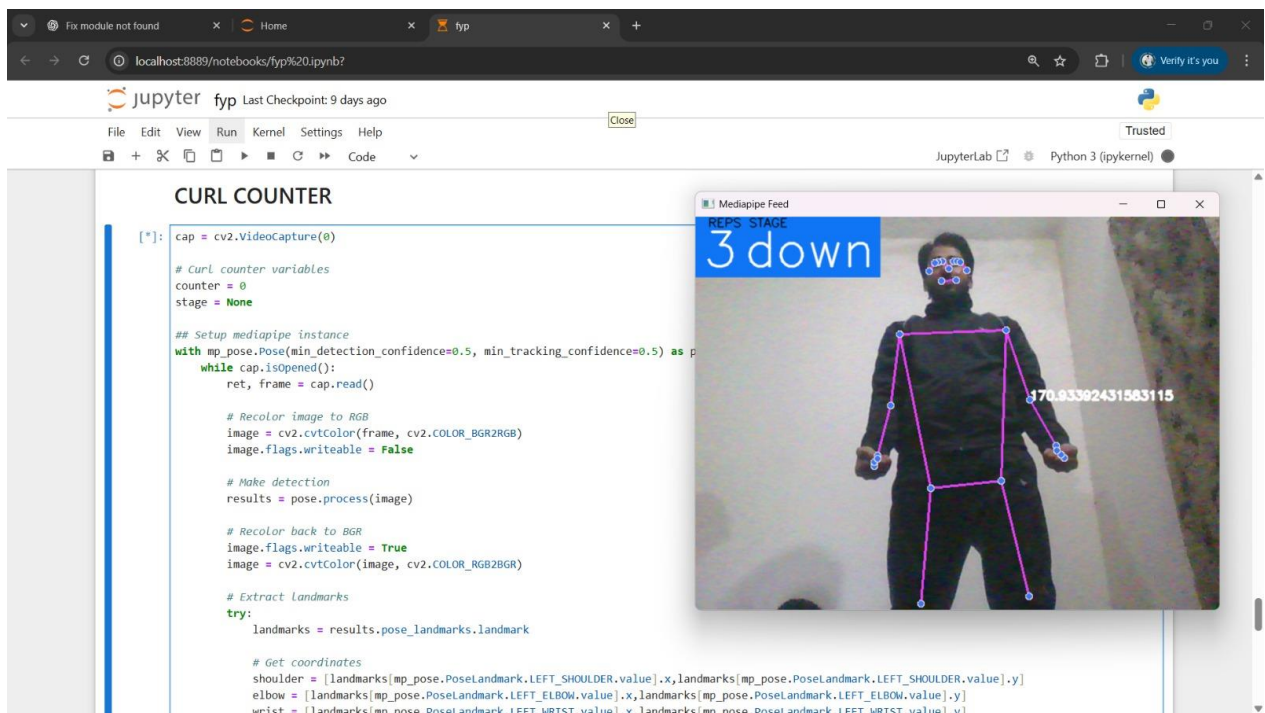
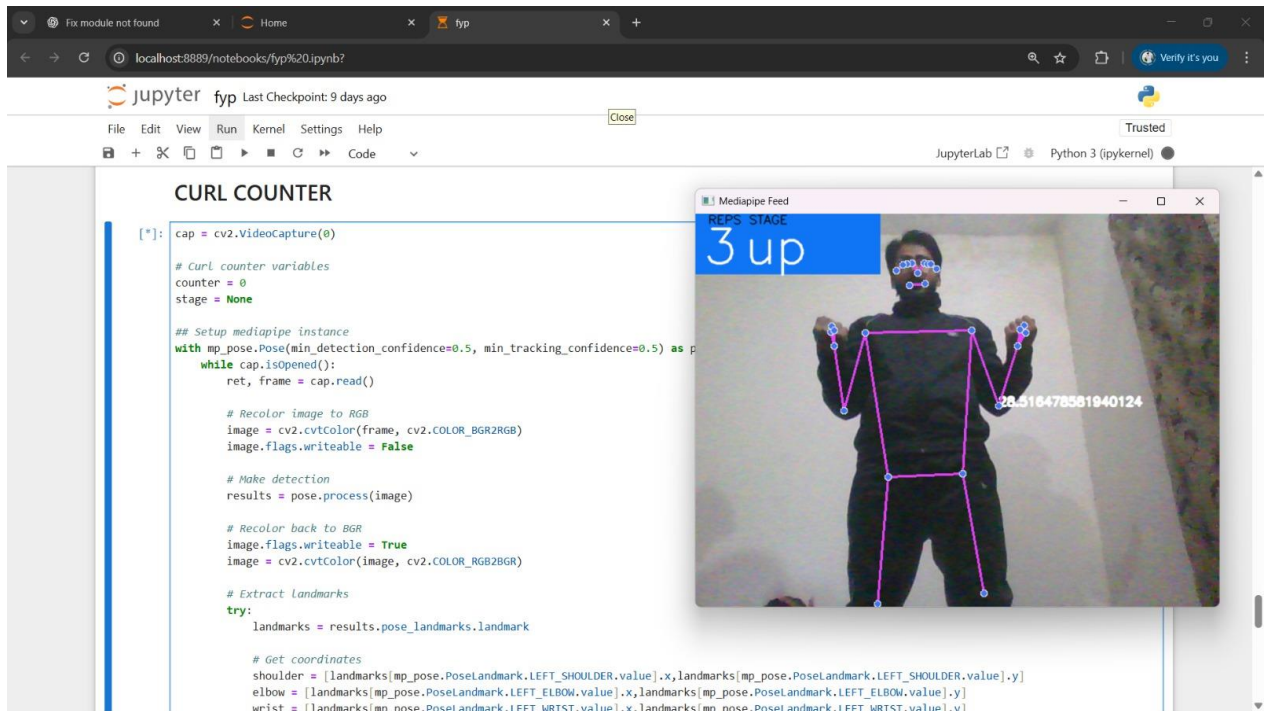
    # If frame not read correctly, skip or stop
    if not ret or frame is None:
        print("⚠ Warning: Empty frame received. Trying again...")
        continue # use 'break' if you want to stop instead

    # Display frame
    cv2.imshow('Mediapipe Feed', frame)

    # Exit when 'q' is pressed
    if cv2.waitKey(10) & 0xFF == ord('q'):
        print("⏏ Exiting...")
        break

# Release and close all windows
cap.release()
cv2.destroyAllWindows()
```





References:

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Upper Saddle River, NJ: Pearson Education, 2021.
- [2] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. London: Springer, 2022.
- [3] A. Howard et al., “MediaPipe: A Framework for Building Perception Pipelines,” *arXiv preprint arXiv:1906.08172*, 2019.
- [4] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’s Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, Nov. 2000.
- [5] Z. Cao, T. Simon, S. E. Wei, and Y. Sheikh, “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 1, pp. 172–186, Jan. 2021.
- [6] T. Pfister, J. Charles, and A. Zisserman, “Flowing ConvNets for Human Pose Estimation in Videos,” in *Proc. IEEE Int. Conf. Computer Vision*, Santiago, Chile, 2015, pp. 1913–1921.
- [7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [8] Google, “MediaPipe Pose Solution,” 2023 [Online]. Available: <https://developers.google.com/mediapipe/solutions/pose>
- [9] OpenCV Team, “OpenCV Documentation,” 2023 [Online]. Available: <https://docs.opencv.org/>
- [10] M. Javaid et al., “Computer Vision and Artificial Intelligence Enabled Fitness Monitoring Systems: A Review,” *Journal of Ambient Intelligence and Humanized Computing*, vol. 12, no. 6, pp. 6201–6214, Jun. 2021.
- [11] Mozilla Developer Network, “JavaScript Guide,” 2023 [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

[12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097–1105, 2012.

Appendix:

The Appendix section contains supplementary information that supports the **AI-Based Gym Trainer System** but is too detailed to be included in the main body of the report. Including such information in the main chapters could burden the reader or distract from the core discussion. Therefore, this section is provided to enhance understanding while maintaining clarity and readability of the main document. All materials included in the appendices are easily presentable in print format and are referenced where necessary.

Appendix A: List of Supported Exercises (Current Phase)

This appendix provides a detailed list of exercises supported by the system in the current phase of development.

- Bicep Curl (Upward and Downward Movement)
- Elbow Angle Measurement for Bicep Curl

Note: Additional exercises will be included in future phases of the project.

Appendix B: Detailed Description of Pose Detection Process

This appendix explains the pose detection workflow used in the system:

- Real-time video capture through webcam
 - Body landmark detection using MediaPipe Pose
 - Drawing skeletal lines and joints using OpenCV
 - Calculation of elbow joint angles
 - Logic used for repetition counting based on angle thresholds
-

Appendix C: System Tools and Libraries

This appendix lists the software tools and libraries used in the development of the project:

- Python Programming Language
 - MediaPipe Pose Library
 - OpenCV (Computer Vision Library)
 - JavaScript (Frontend Development)
 - Web Browser (for application execution)
-

Appendix D: Sample Output Screenshots

This appendix includes screenshots of the application interface developed so far:

- Webcam activation screen
- Real-time pose landmark visualization
- Display of joint angle values
- Exercise repetition counter display

(Screenshots are attached separately in the printed version of the report.)

Appendix E: Supporting Literature

This appendix provides a list of relevant articles and resources that support the project domain but are not directly cited within the main chapters:

- Research articles on AI-based fitness monitoring systems
- Studies on human pose estimation and exercise analysis
- Documentation of MediaPipe and OpenCV libraries